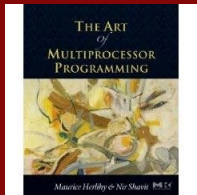
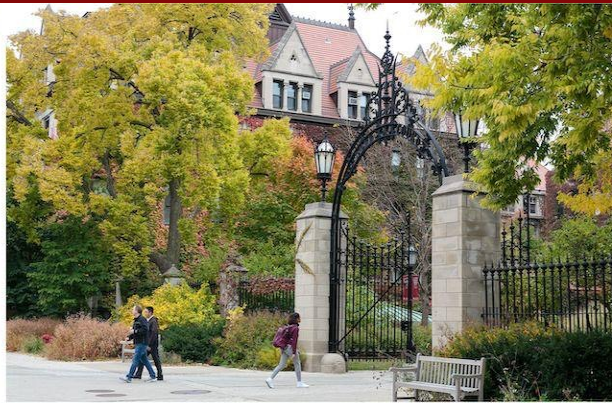


MPCS 52060 - Parallel Programming

M8: Advanced Parallel Patterns and Techniques (Part 2)



Original slides from “The Art of Multiprocessor Programming by Maurice Herlihy & Nir Shavit” With modifications by Lamont Samuels and Jan Hückelheim

Outline

- In previous weeks, we have seen a number of synchronization tools:
 - Locks
 - Condition variables
 - Semaphores
 - Waitgroups
 - Channels
- Today we will learn about another one.

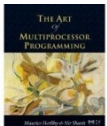
Barriers



- A little like a relay race:
- One stage must complete before the next can start

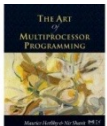
Simple Video Game

- Prepare frame for display
 - By graphics coprocessor
- “soft real-time” application
 - Need at least 35 frames/second
 - OK to mess up rarely



Simple Video Game

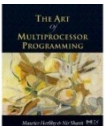
```
while (true) {  
    frame.prepare();  
    frame.display();  
}
```



Simple Video Game

```
while (true) {  
    frame.prepare();  
    frame.display();  
}
```

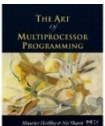
- What about overlapping work?
 - 1st thread displays frame
 - 2nd prepares next frame



Two-Phase Rendering

```
while (true) {  
  if (phase) {  
    frame[0].display();  
  } else {  
    frame[1].display();  
  }  
  phase = !phase;  
}
```

```
while (true) {  
  if (phase) {  
    frame[1].prepare();  
  } else {  
    frame[0].prepare();  
  }  
  phase = !phase;  
}
```

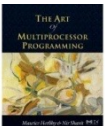


Two-Phase Rendering

```
while (true) {  
  if (phase) {  
    frame[0].display();  
  } else {  
    frame[1].display();  
  }  
  phase = !phase;  
}
```

```
while (true) {  
  if (phase) {  
    frame[1].prepare();  
  } else {  
    frame[0].prepare();  
  }  
  phase = !phase;  
}
```

even phases

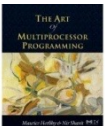


Two-Phase Rendering

```
while (true) {  
  if (phase) {  
    frame[0].display();  
  } else {  
    frame[1].display();  
  }  
  phase = !phase;  
}
```

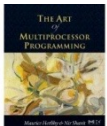
```
while (true) {  
  if (phase) {  
    frame[1].prepare();  
  } else {  
    frame[0].prepare();  
  }  
  phase = !phase;  
}
```

odd phases

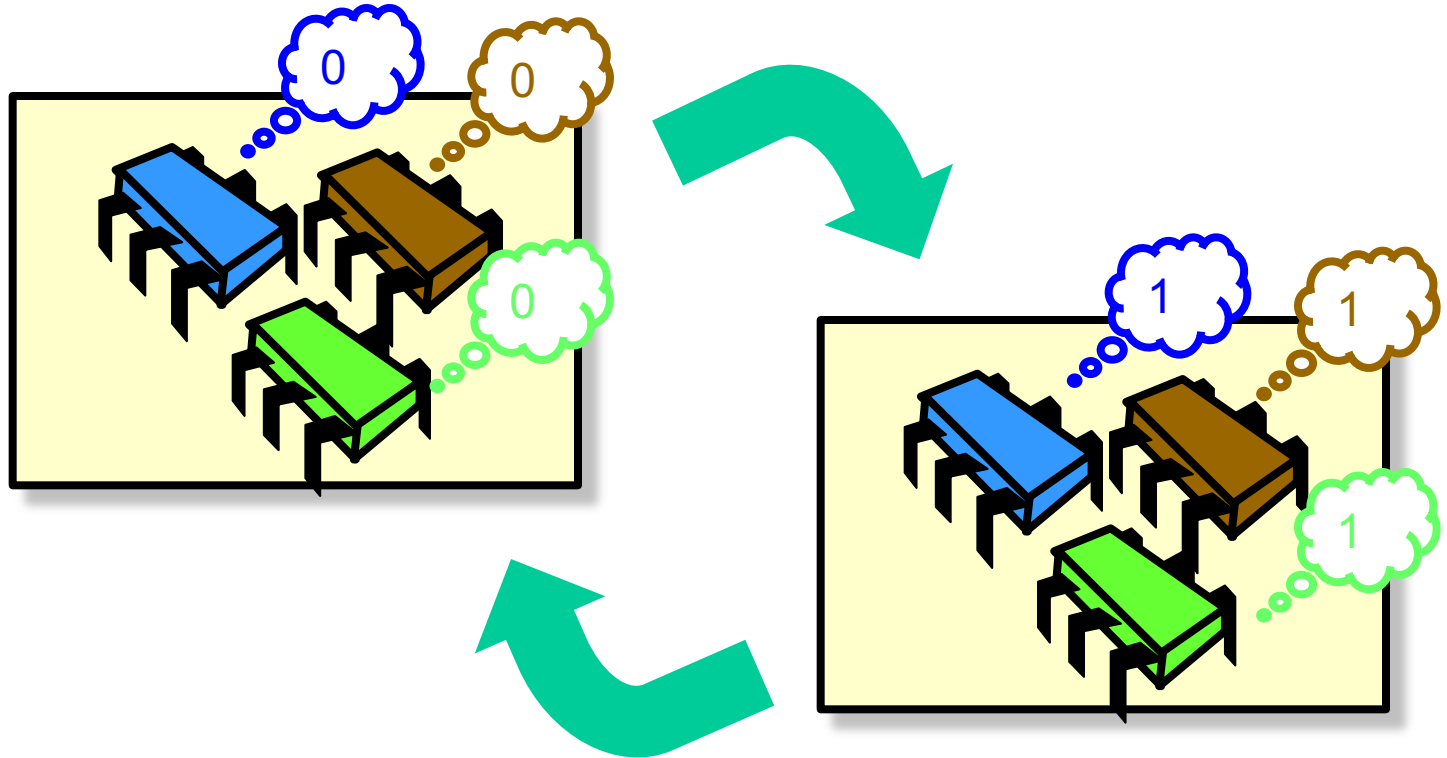


Synchronization Problems

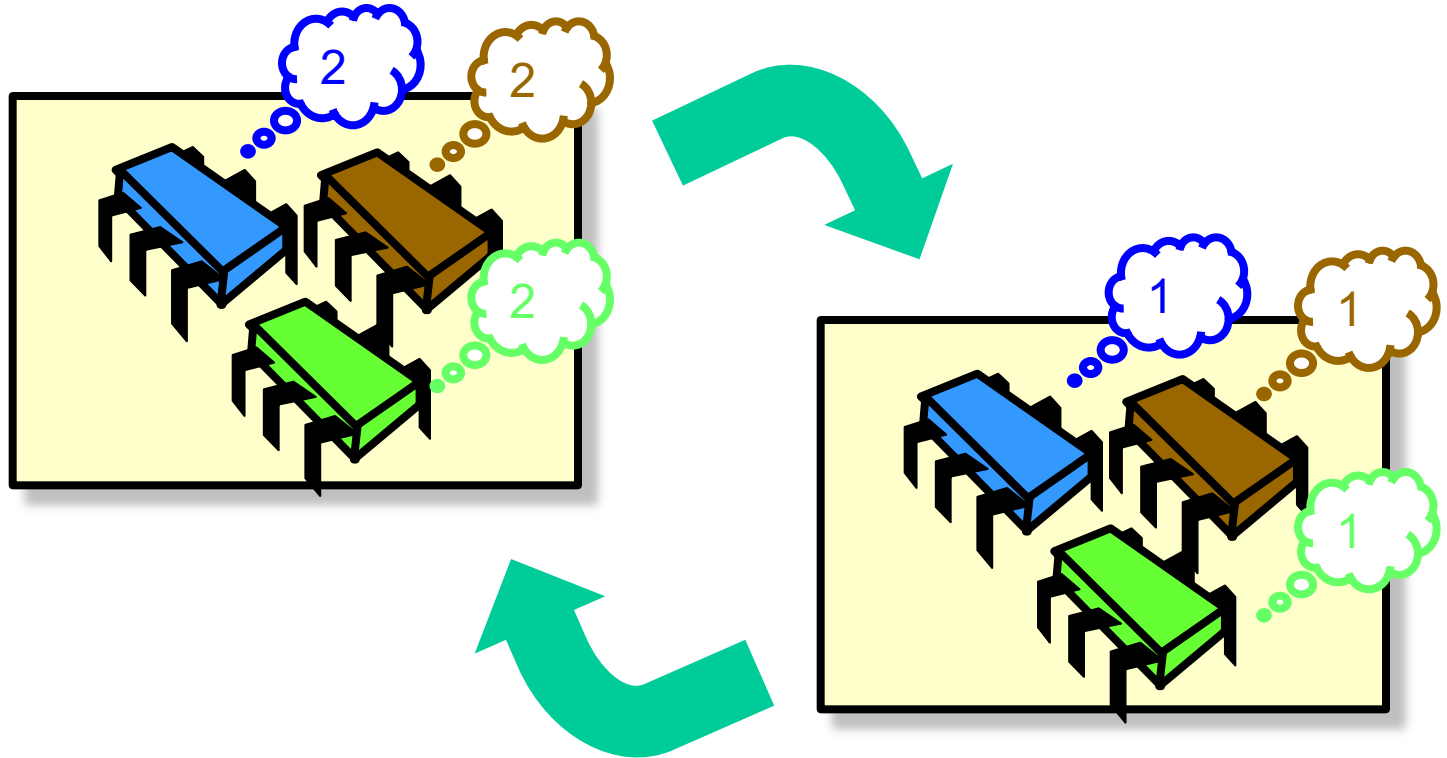
- How do threads stay in phase?
- Too early?
 - “we render no frame before its time”
- Too late?
 - Recycle memory before frame is displayed



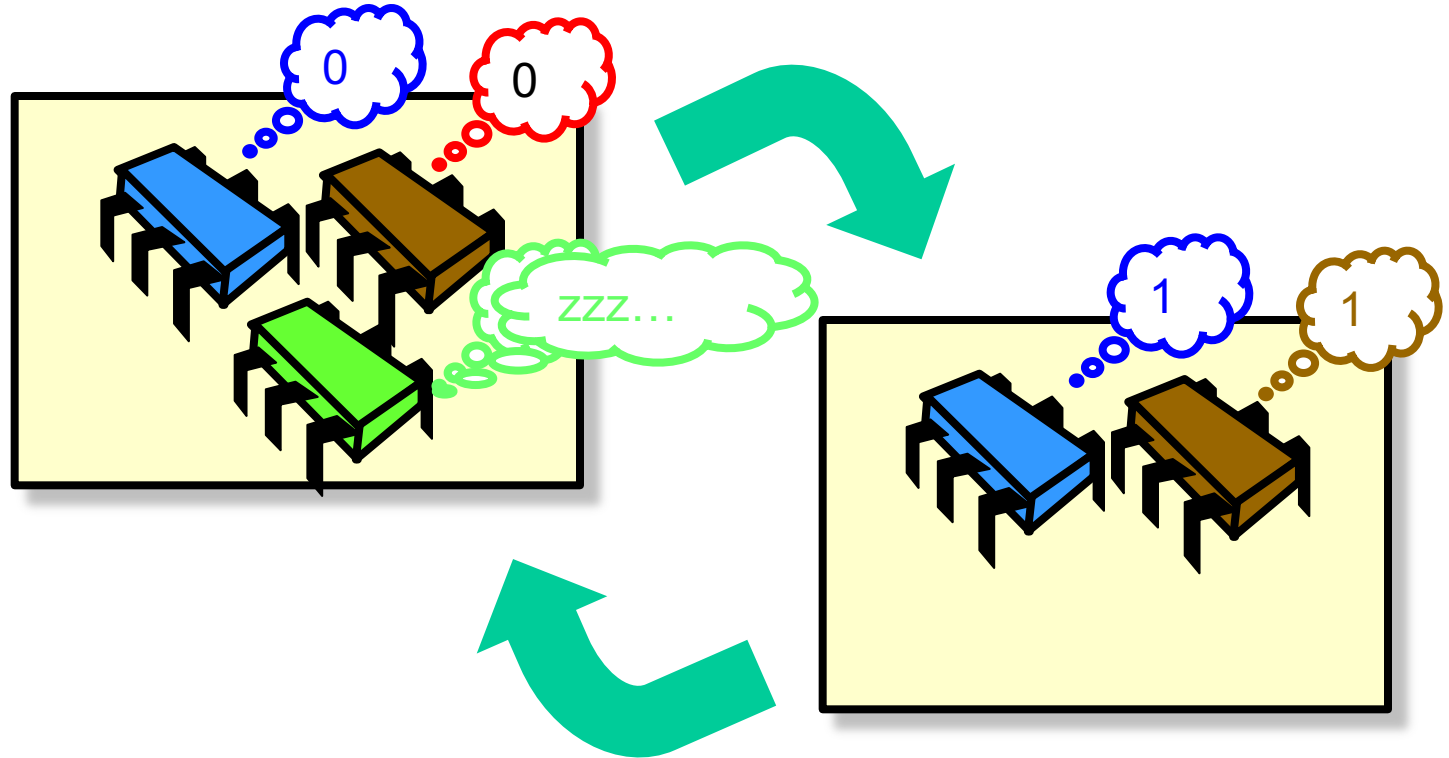
Ideal Parallel Computation



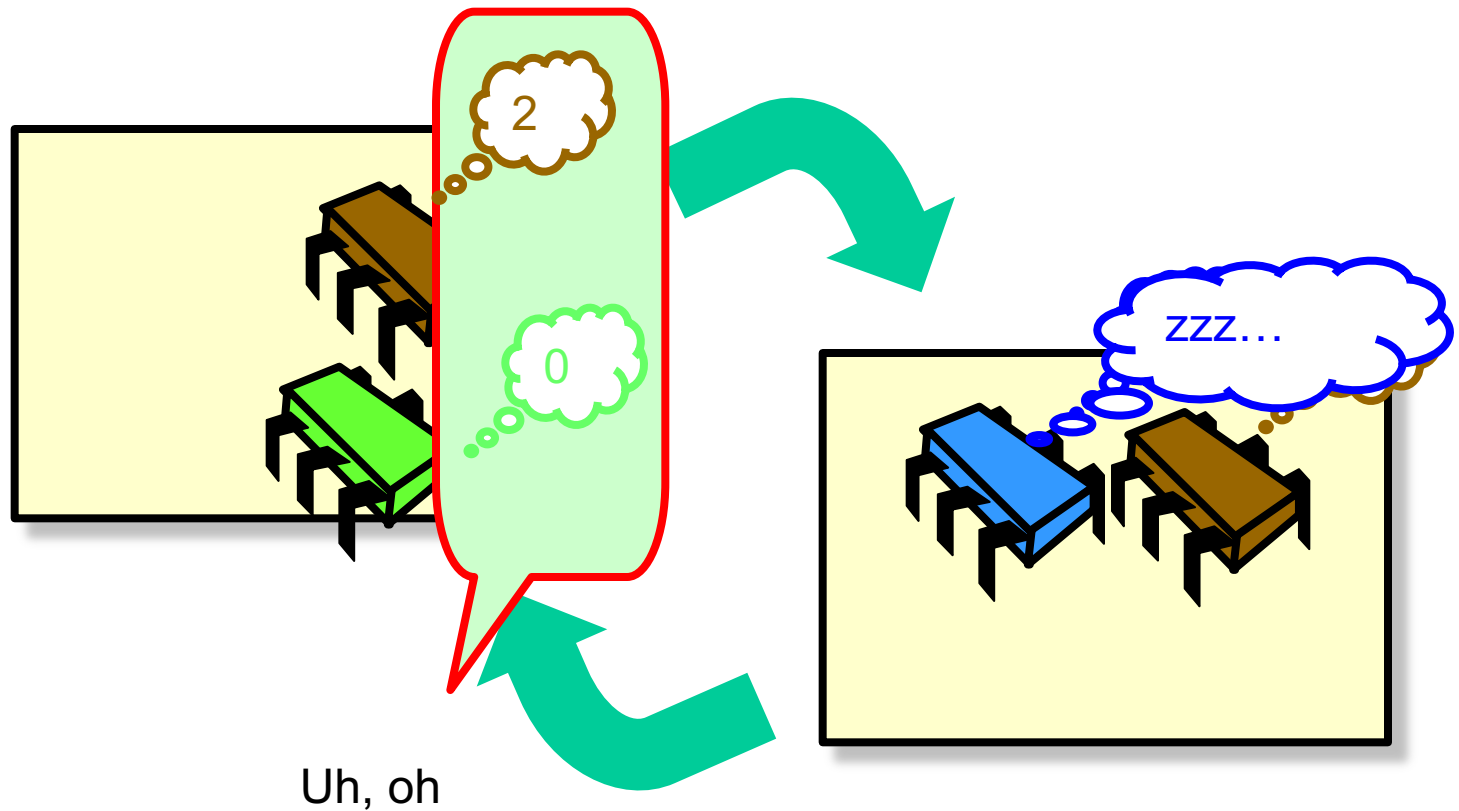
Ideal Parallel Computation



Real-Life Parallel Computation



Real-Life Parallel Computation



Another way to put it...

https://www.reddit.com/r/aww/comments/2oagj8/multithreaded_programming_theory_and_practice/



THE UNIVERSITY OF
CHICAGO

**MASTERS PROGRAM
IN COMPUTER SCIENCE**

Another way to put it...

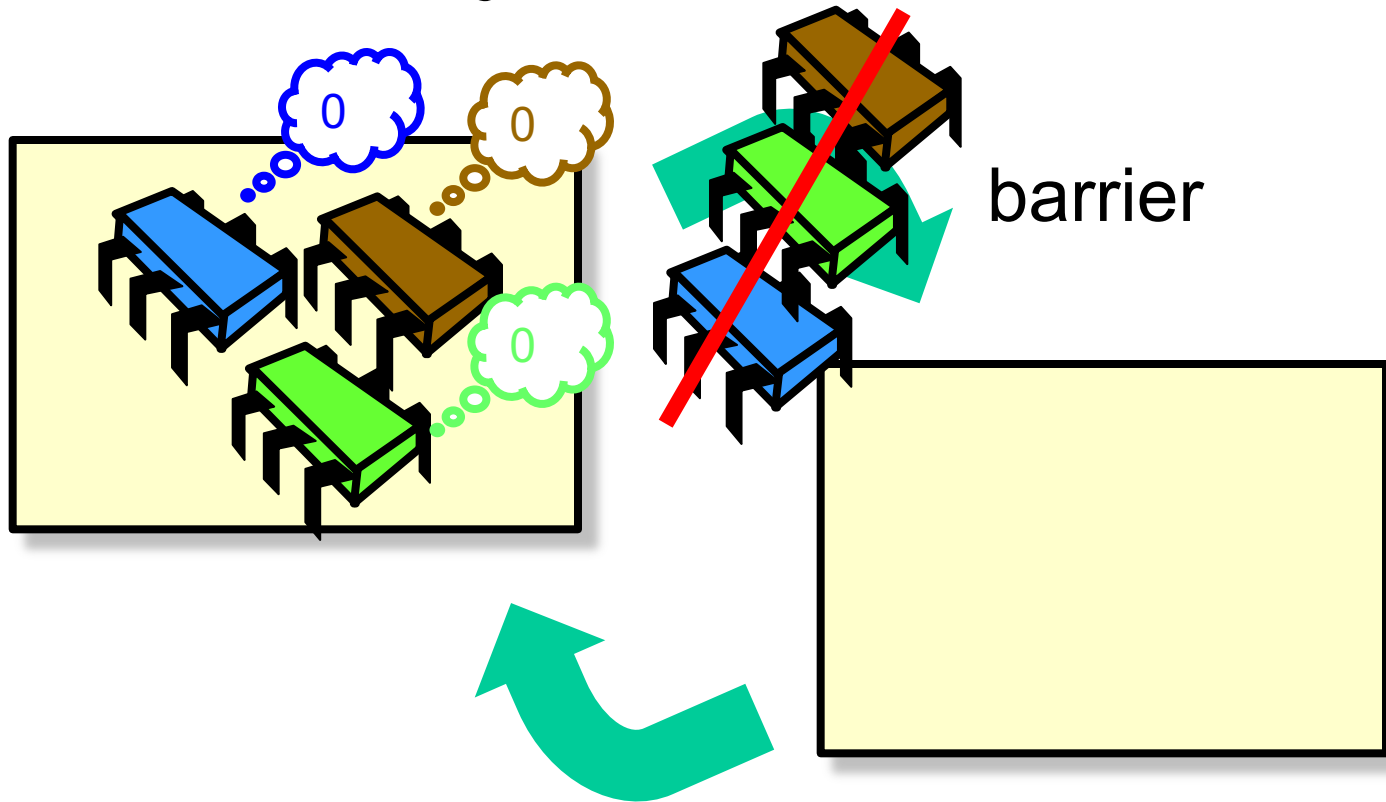
https://www.reddit.com/r/aww/comments/2oagj8/multithreaded_programming_theory_and_practice/



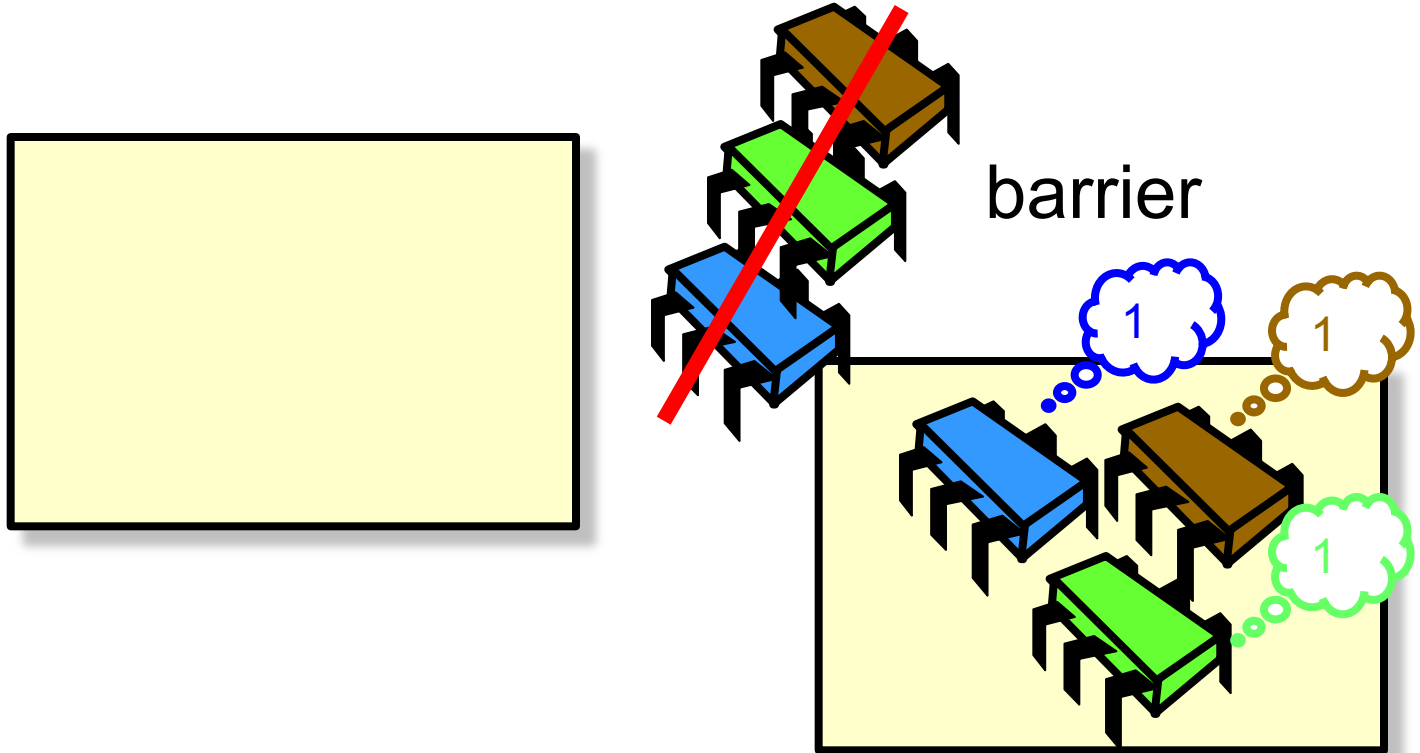
THE UNIVERSITY OF
CHICAGO

**MASTERS PROGRAM
IN COMPUTER SCIENCE**

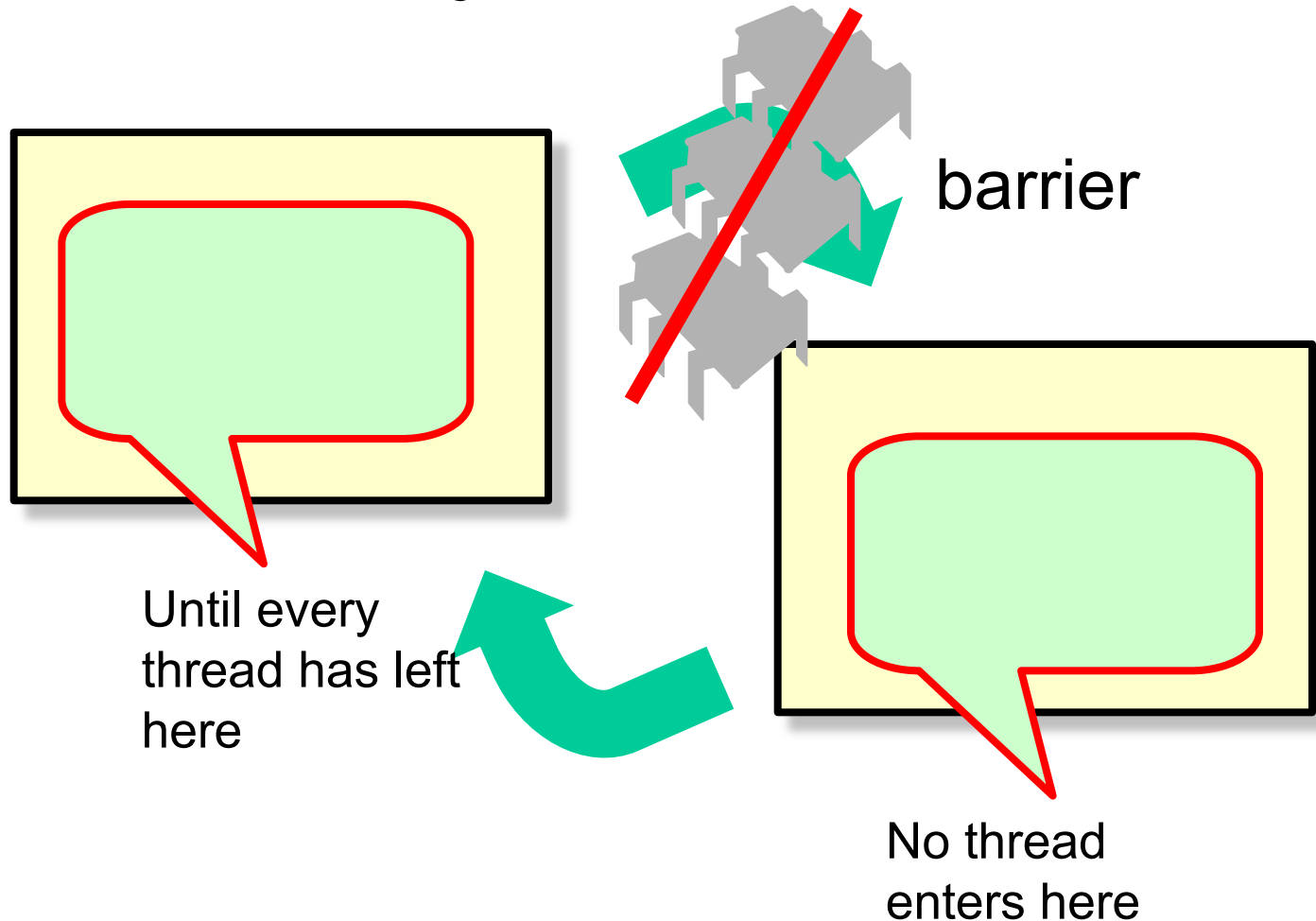
Barrier Synchronization



Barrier Synchronization

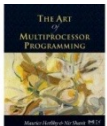


Barrier Synchronization



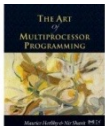
Why Do We Care?

- Mostly of interest to
 - Scientific & numeric computation
- Elsewhere
 - Garbage collection
 - Less common in systems programming
 - Still important topic

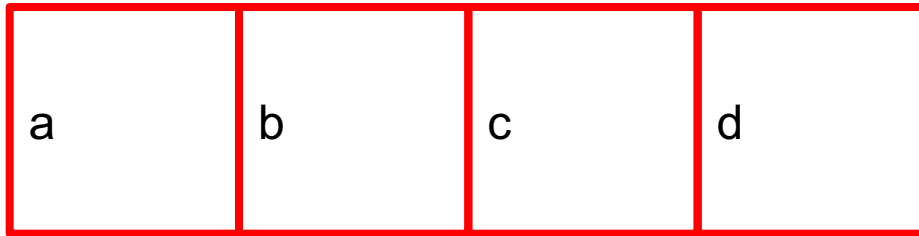


Duality

- Dual to mutual exclusion
 - Include others, not exclude them
- Same implementation issues
 - Interaction with caches ...
 - Invalidation?
 - Local spinning?

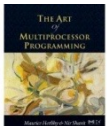
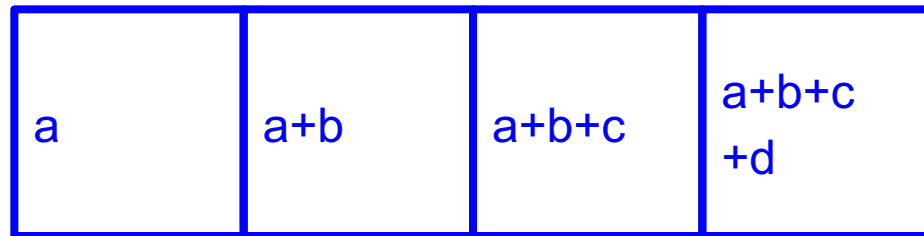


Example: Parallel Prefix



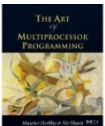
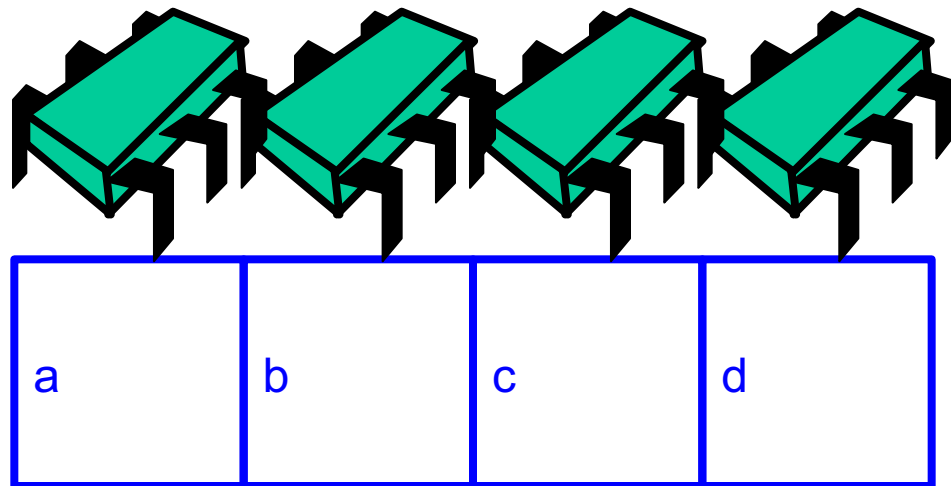
before

after

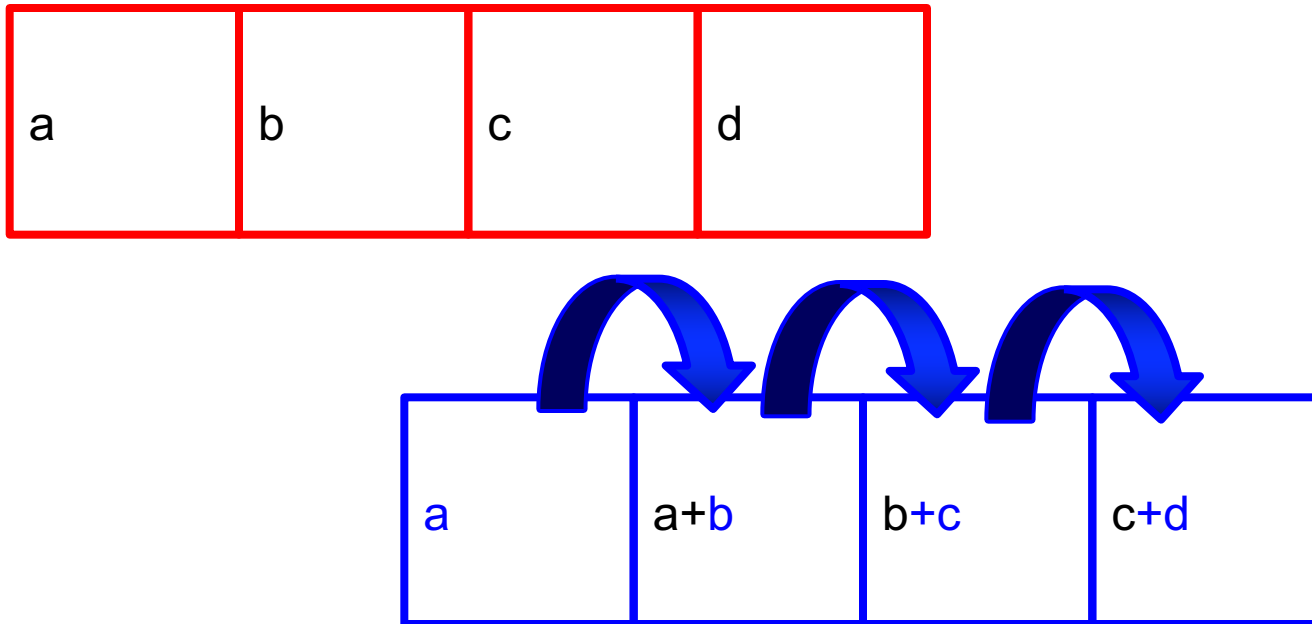


Parallel Prefix

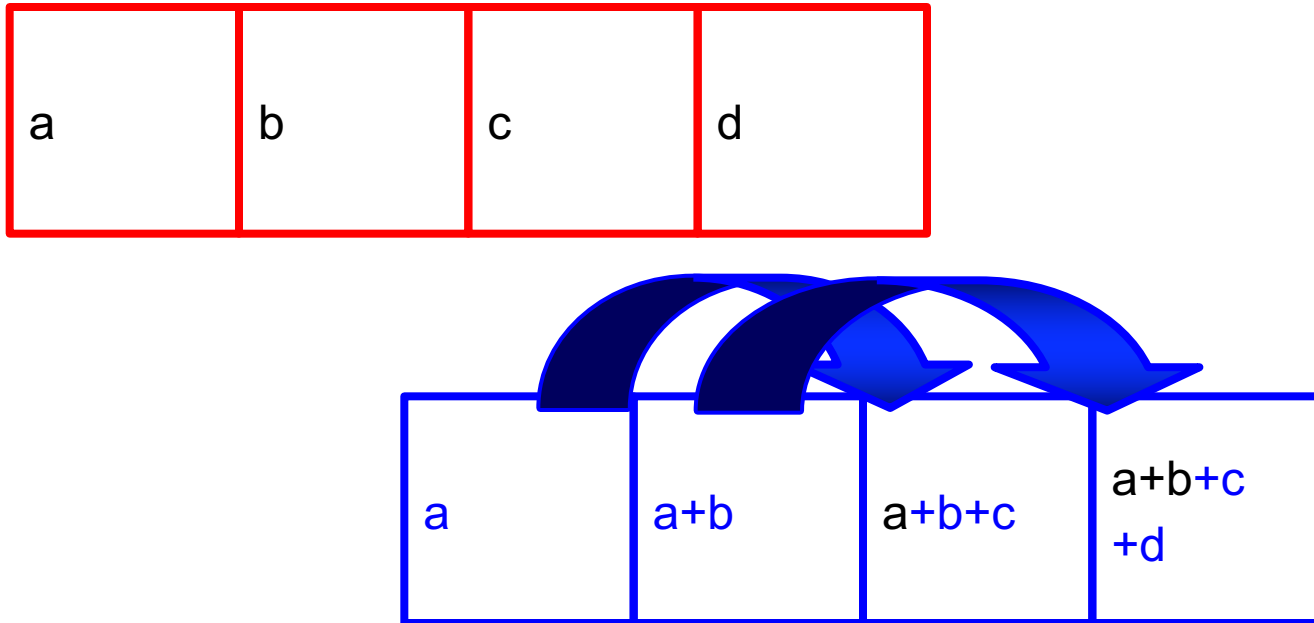
One thread
Per entry



Parallel Prefix: Phase 1

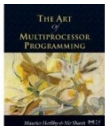


Parallel Prefix: Phase 2



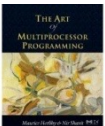
Parallel Prefix

- N threads can compute
 - Parallel prefix
 - Of N entries
 - In $\log_2 N$ rounds
- What if system is asynchronous?
 - Why we need barriers



Prefix

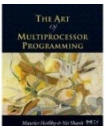
```
class Prefix extends Thread {  
    int[] a;  
    int i;  
    Barrier b;  
    void Prefix(int[] a,  
                Barrier b, int i) {  
  
        a = a;  
        b = b;  
        i = i;  
    }  
}
```



Prefix

```
class Prefix extends Thread {  
    int[] a;  
    int i;  
    Barrier b;  
    void Prefix(int[] a,  
                Barrier b, int i) {  
        a = a;  
        b = b;  
        i = i;  
    }  
}
```

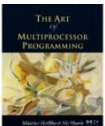
Array of input values



Prefix

```
class Prefix extends Thread {  
    int[] a;  
    int i;  
    Barrier b;  
    void Prefix(int[] a,  
                Barrier b, int i) {  
  
        a = a;  
        b = b;  
        i = i;  
    }  
}
```

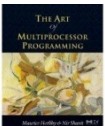
Thread index



Prefix

```
class Prefix extends Thread {  
    int[] a;  
    int i;  
    Barrier b;  
    void Prefix(int[] a,  
                Barrier b, int i) {  
        a = a;  
        b = b;  
        i = i;  
    }  
}
```

Shared barrier

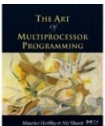


Prefix

```
class Prefix extends Thread {  
    int[] a;  
    int i;  
    Barrier b;  
    void Prefix(int[] a,  
                Barrier b, int i) {
```

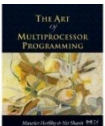
Initialize fields

```
    a = a;  
    b = b;  
    i = i;  
}
```



Where Do the Barriers Go?

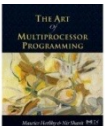
```
public void run() {  
    int d = 1, sum = 0;  
    while (d < N) {  
        if (i >= d)  
            sum = a[i-d];  
        if (i >= d)  
            a[i] += sum;  
        d = d * 2;  
    }  
}
```



Where Do the Barriers Go?

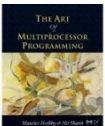
```
public void run() {  
    int d = 1, sum = 0;  
    while (d < N) {  
        if (i >= d)  
            sum = a[i-d];  
        b.await();  
        if (i >= d)  
            a[i] += sum;  
        d = d * 2;  
    }  
}
```

Make sure everyone reads before anyone writes



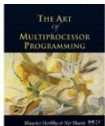
Where Do the Barriers Go?

```
public void run() {  
    int d = 1, sum = 0;  
    while (d < N) {  
        if (i >= d)  
            sum = a[i-d];  
        b.await();           Make sure everyone reads before anyone  
                             writes  
        if (i >= d)  
            a[i] += sum;  
        b.await();           Make sure everyone writes before anyone  
                             reads  
        d = d * 2;  
    }  
}
```



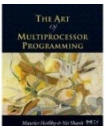
Barrier Implementations

- Cache coherence
 - Spin on locally-cached locations?
 - Spin on statically-defined locations?
- Latency
 - How many steps?
- Symmetry
 - Do all threads do the same thing?



Barriers

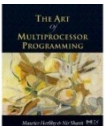
```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

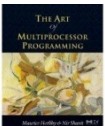
Number of threads not yet
arrived



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n) {  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

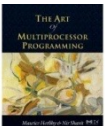
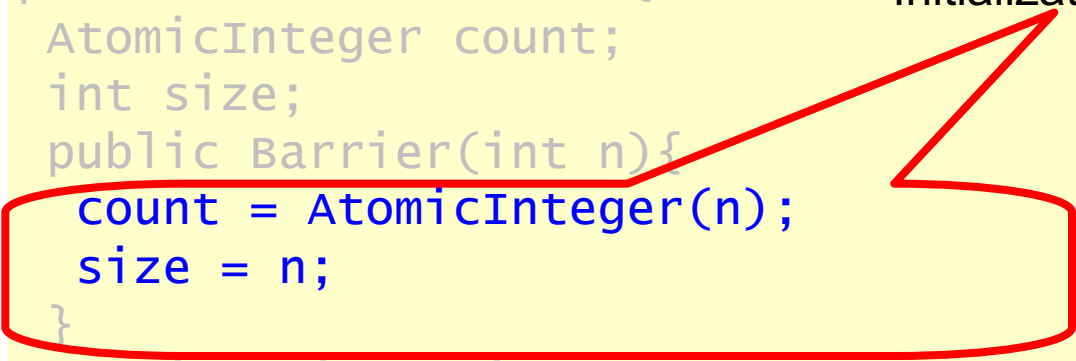
Number of threads participating



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

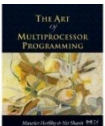
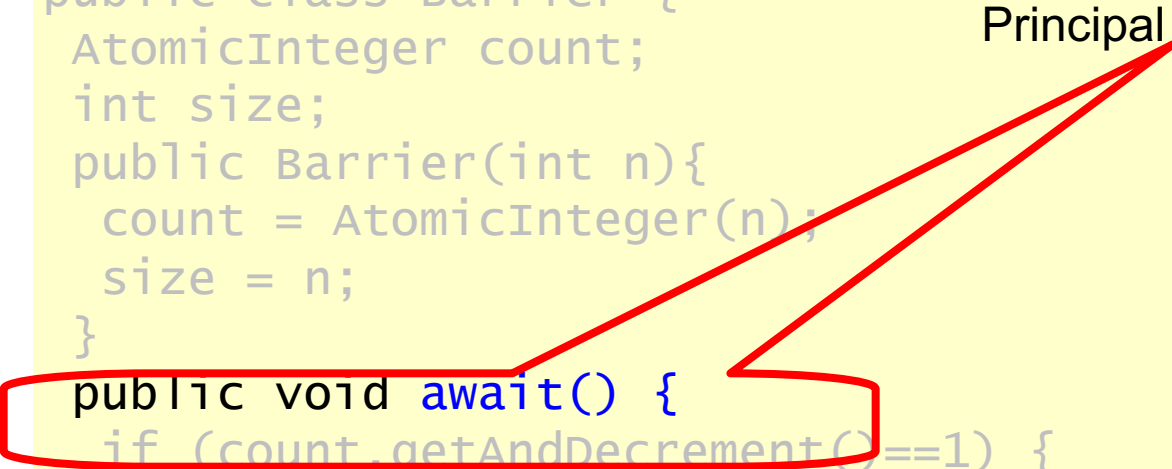
Initialization



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement() == 1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

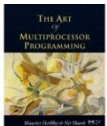
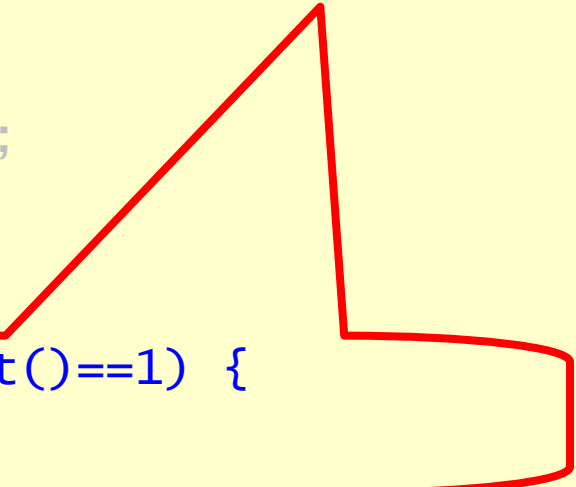
Principal method



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

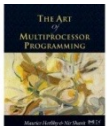
If I'm last, reset fields for next time



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

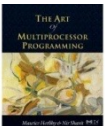
Otherwise, wait for
everyone else



Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement() == 1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

What's wrong with this protocol?



Reuse

```
Barrier b = new Barrier(n);
```

```
while ( mumble() ) {
```

```
    work();
```

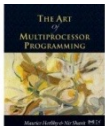
do work

```
    b.await();
```

synchronize

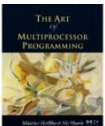
repeat

```
}
```



Barriers

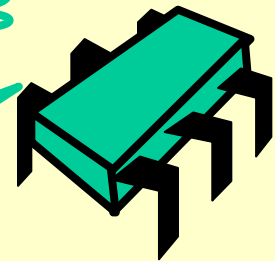
```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```



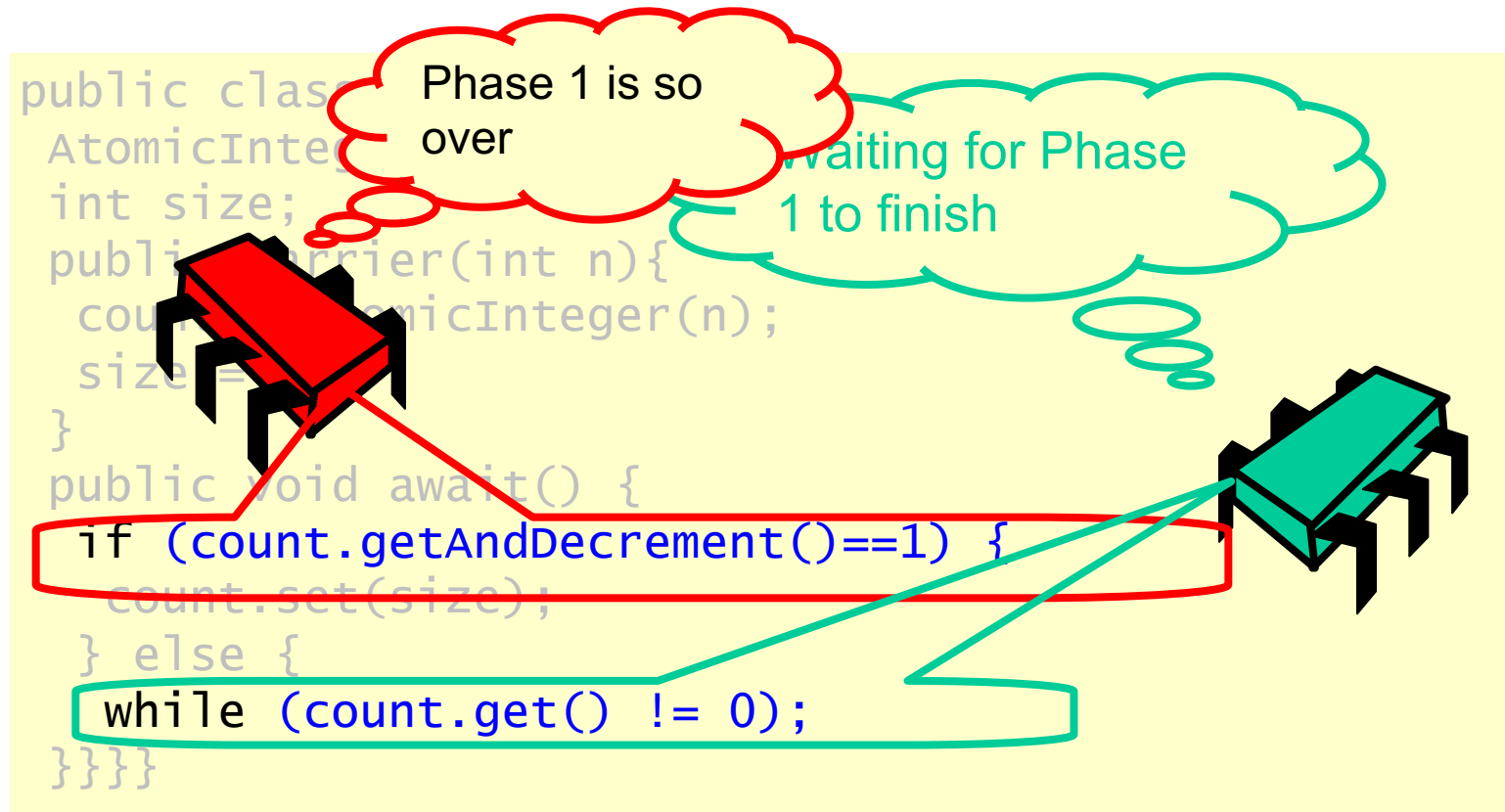
Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n){  
        count = AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

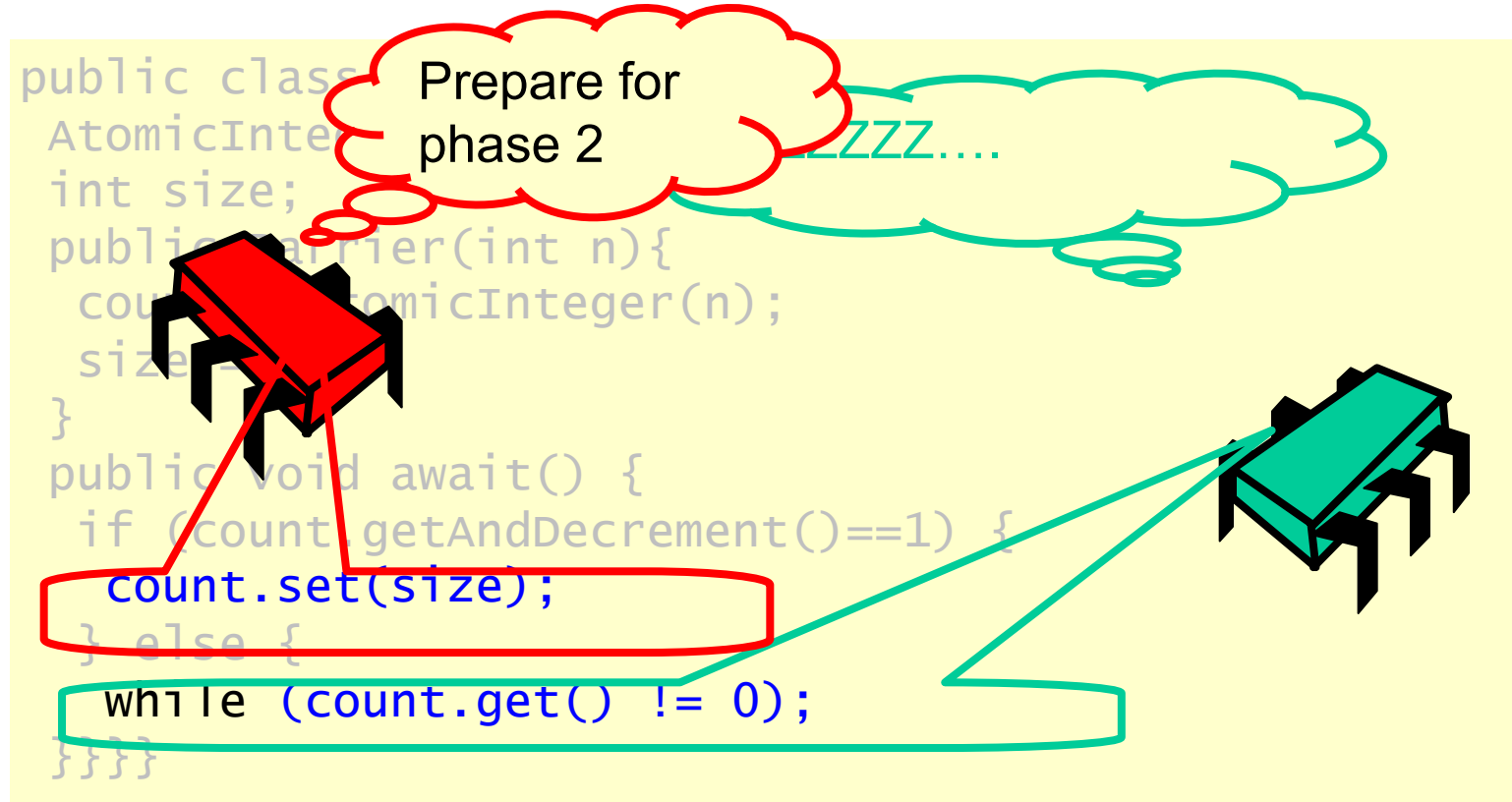
Waiting for Phase
1 to finish



Barriers



Barriers



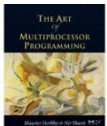
Uh-Oh

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    public Barrier(int n)  
    {  
        count = new AtomicInteger(n);  
        size = n;  
    }  
    public void await() {  
        if (count.getAndDecrement() == 1) {  
            count.set(size);  
        } else {  
            while (count.get() != 0);  
        }  
    }  
}
```

Waiting for Phase
2 to finish

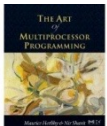
Waiting for Phase
1 to finish

while (count.get() != 0);



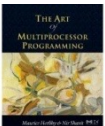
Basic Problem

- One thread “wraps around” to start phase 2
- While another thread is still waiting for phase 1
- One solution:
 - Always use two barriers



Sense-Reversing Barriers

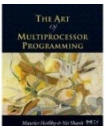
```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    boolean sense = false;  
    threadSense = new ThreadLocal<boolean>...  
  
    public void await {  
        boolean mySense = threadSense.get();  
        if (count.getAndDecrement()==1) {  
            count.set(size); sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
        threadSense.set(!mySense)}}  
    }
```



Sense-Reversing Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    boolean sense = false;  
    ThreadLocal<boolean> threadSense = new ThreadLocal<boolean>...  
  
    public void await {  
        boolean mySense = threadSense.get();  
        if (count.getAndDecrement()==1) {  
            count.set(size); sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
        threadSense.set(!mySense)}}}
```

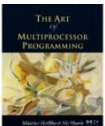
Completed odd or even-numbered phase?



Sense-Reversing Barriers

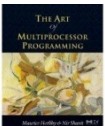
```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    boolean sense = false;  
    threadSense = new ThreadLocal<boolean>...  
  
    public void await {  
        boolean mySense = threadSense.get();  
        if (count.getAndDecrement()==1) {  
            count.set(size); sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
        threadSense.set(!mySense)}}}
```

Store sense for next phase



Sense-Reversing Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;                Get new sense determined by last phase  
    boolean sense = false;  
    threadSense = new ThreadLocal<boolean>...  
  
    public void await {  
        boolean mySense = threadSense.get();  
        if (count.getAndDecrement() == 1) {  
            count.set(size); sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
        threadSense.set(!mySense)}}}
```

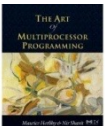


Sense-Reversing Barriers

```
public class Barrier {
    AtomicInteger count;
    int size;
    boolean sense = false;
    ThreadLocal<boolean> threadSense = new ThreadLocal<boolean>();

    public void await {
        boolean mySense = threadSense.get();
        if (count.getAndDecrement()==1) {
            count.set(size); sense = mySense
        } else {
            while (sense != mySense) {}
        }
        threadSense.set(!mySense)}}}
```

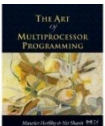
If I'm last, reverse sense for next time



Sense-Reversing Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    boolean sense = false;  
    ThreadLocal<boolean> threadSense = new ThreadLocal<boolean>();  
  
    public void await {  
        boolean mySense = threadSense.get();  
        if (count.getAndDecrement() == 1) {  
            count.set(size); sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
        threadSense.set(!mySense);  
    }  
}
```

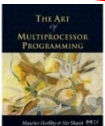
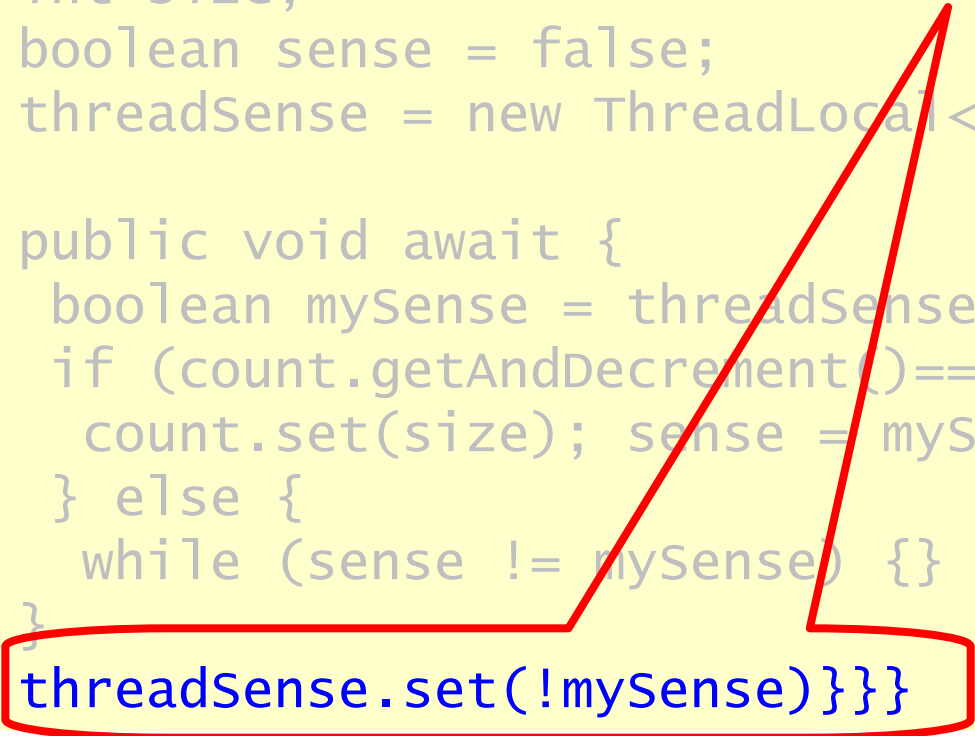
Otherwise, wait for sense to flip



Sense-Reversing Barriers

```
public class Barrier {  
    AtomicInteger count;  
    int size;  
    boolean sense = false;  
    ThreadLocal<boolean> threadSense = new ThreadLocal<boolean>();  
  
    public void await () {  
        boolean mySense = threadSense.get();  
        if (count.getAndDecrement() == 1) {  
            count.set(size); sense = mySense;  
        } else {  
            while (sense != mySense) {}  
        }  
        threadSense.set(!mySense);  
    }  
}
```

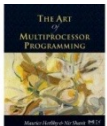
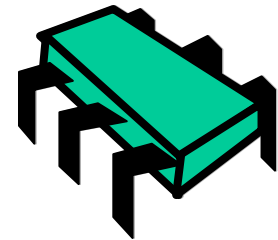
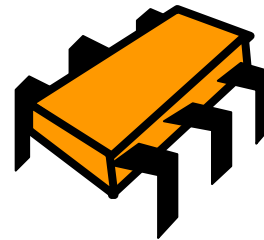
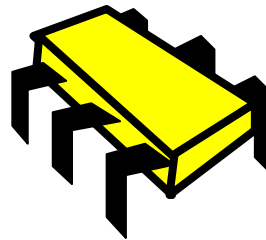
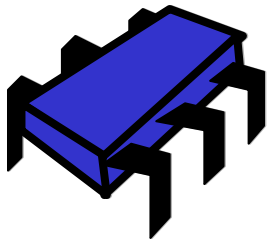
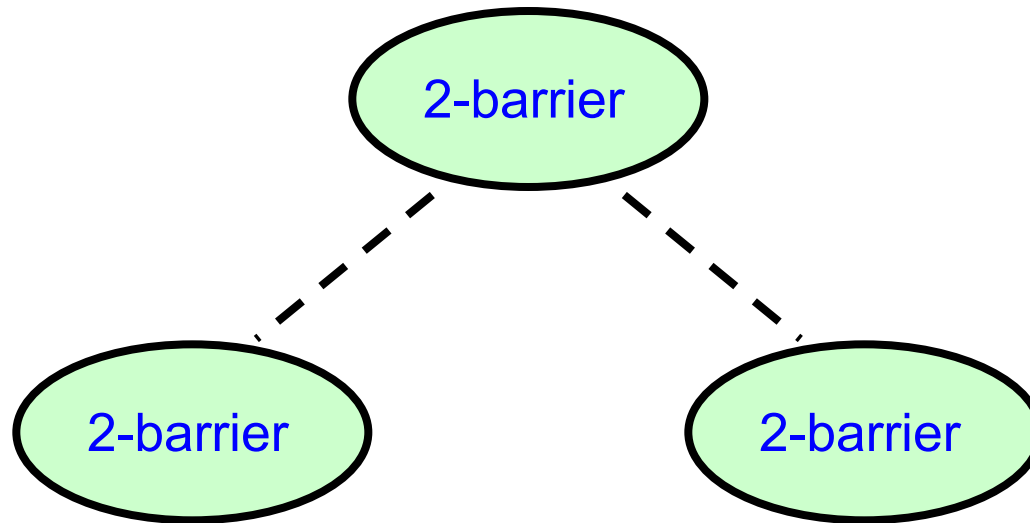
Prepare sense for next phase



Is this a good barrier?

- It works
- It is simple
- But: all threads use the same atomic counter

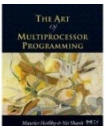
Combining Tree Barriers



Combining Tree Barrier

```
public class Node{
    AtomicInteger count; int size;
    Node parent; volatile boolean sense;

    public void await() {...
        if (count.getAndDecrement()==1) {
            if (parent != null)
                parent.await()
            count.set(size);
            sense = mySense
        } else {
            while (sense != mySense) {}
        }
    }...}}}
```

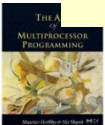


Combining Tree Barrier

```
public class Node{
    AtomicInteger count; int size;
    Node parent; volatile boolean sense;

    public void await() {...
        if (count.getAndDecrement()==1) {
            if (parent != null)
                parent.await()
            count.set(size);
            sense = mySense
        } else {
            while (sense != mySense) {}
        }
    }...}}}
```

Parent barrier in tree

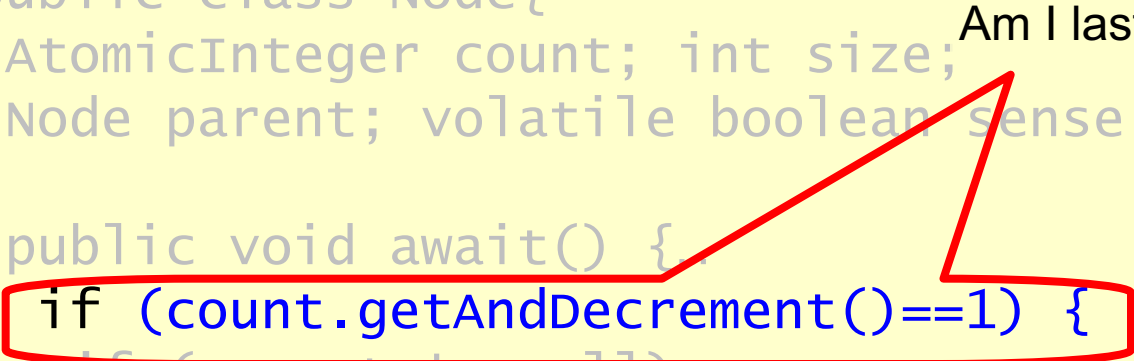


Combining Tree Barrier

```
public class Node{
    AtomicInteger count; int size;
    Node parent; volatile boolean sense;

    public void await() {
        if (count.getAndDecrement()==1) {
            if (parent != null)
                parent.await()
            count.set(size);
            sense = mySense
        } else {
            while (sense != mySense) {}
        }
    }
}
```

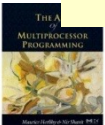
Am I last?



Combining Tree Barrier

```
public class Node {  
    AtomicInteger count; int size;  
    Node parent; volatile boolean sense;  
  
    public void await() {  
        if (count.getAndDecrement() == 1) {  
            if (parent != null)  
                parent.await();  
            count.set(size);  
            sense = mySense;  
        } else {  
            while (sense != mySense) {}  
        }  
    }  
}
```

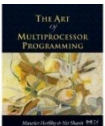
Proceed to parent barrier



Combining Tree Barrier

```
public class Node{  
    AtomicInteger count; int size;  
    Node parent; volatile boolean sense;  
  
    public void await() {  
        if (count.getAndDecrement()==1) {  
            if (parent != null)  
                parent.await()  
            count.set(size);  
            sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
    }  
}
```

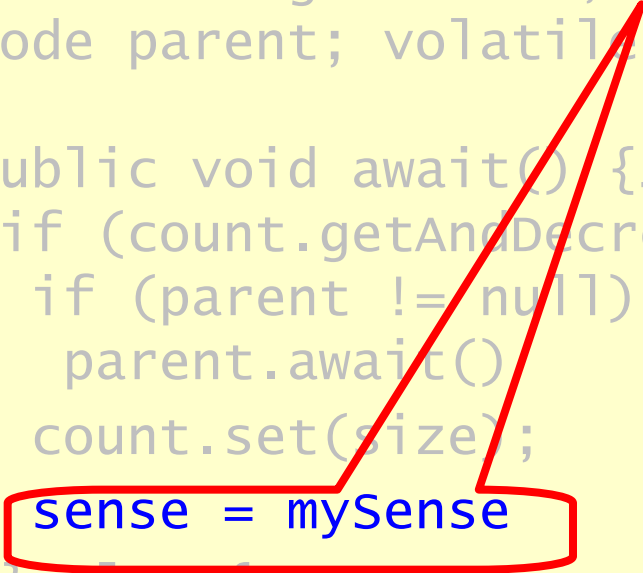
Prepare for next phase



Combining Tree Barrier

```
public class Node {  
    AtomicInteger count; int size;  
    Node parent; volatile boolean sense;  
  
    public void await() {..  
        if (count.getAndDecrement()==1) {  
            if (parent != null)  
                parent.await();  
            count.set(size);  
            sense = mySense  
        } else {  
            while (sense != mySense) {}  
        }  
    }  
}
```

Notify others at this node

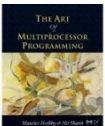
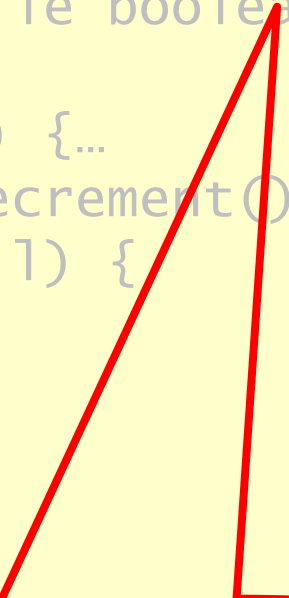


Combining Tree Barrier

```
public class Node{
    AtomicInteger count; int size;
    Node parent; volatile boolean sense;

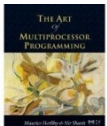
    public void await() {...
        if (count.getAndDecrement()==1) {
            if (parent != null) {
                parent.await()
                count.set(size);
                sense = mySense
            } else {
                while (sense != mySense) {}
            }
        }
    }
}
```

I'm not last, so wait for notification



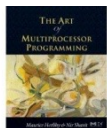
Combining Tree Barrier

- No sequential bottleneck
 - Parallel `getAndDecrement()` calls
- Low memory contention
 - Same reason
- Cache behavior
 - Local spinning on bus-based architecture
 - Not so good for NUMA



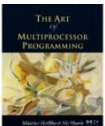
Remarks

- Everyone spins on sense field
 - Local spinning on bus-based (good)
 - Network hot-spot on distributed architecture (bad)
- Not really scalable

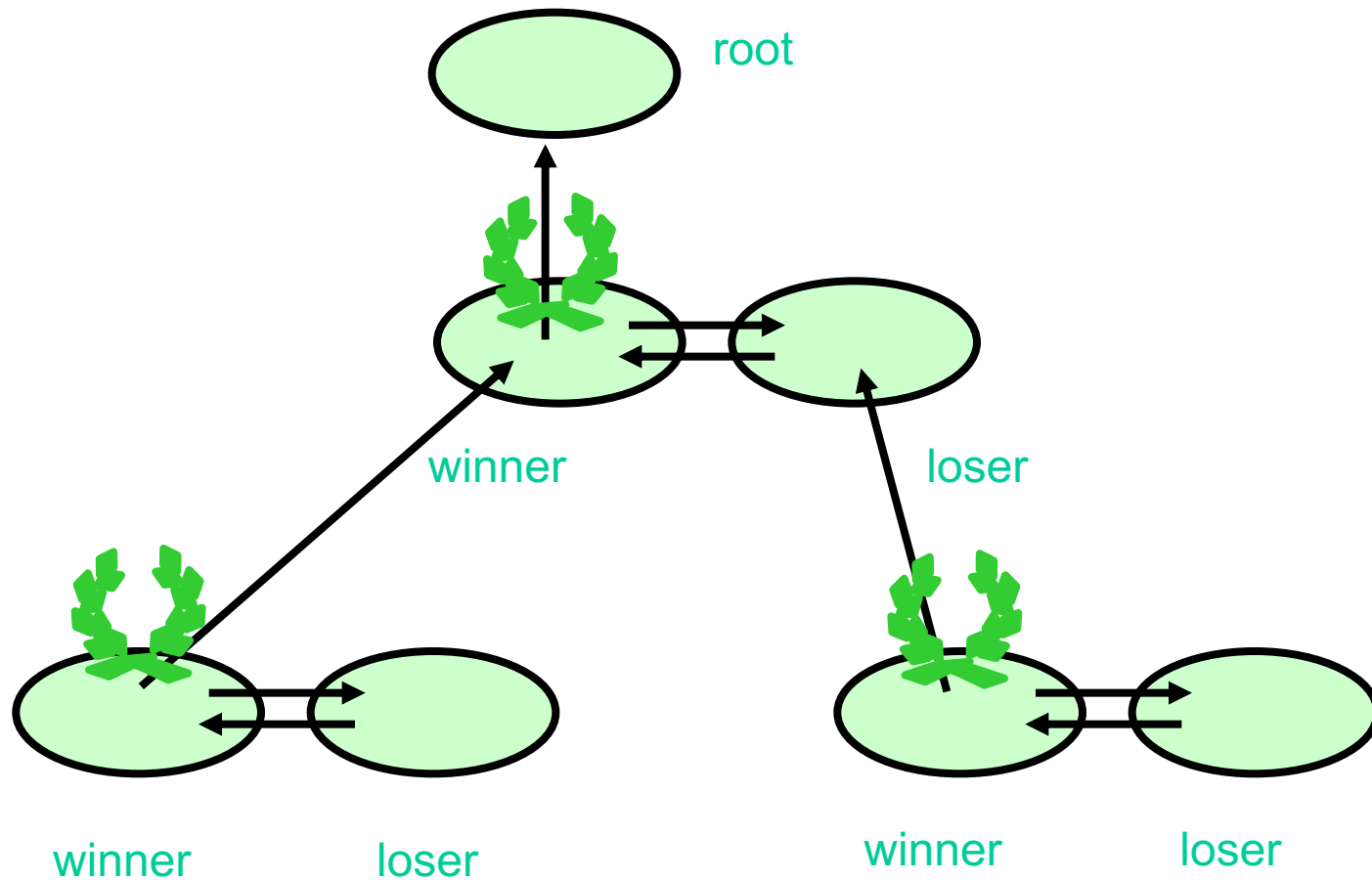


Tournament Tree Barrier

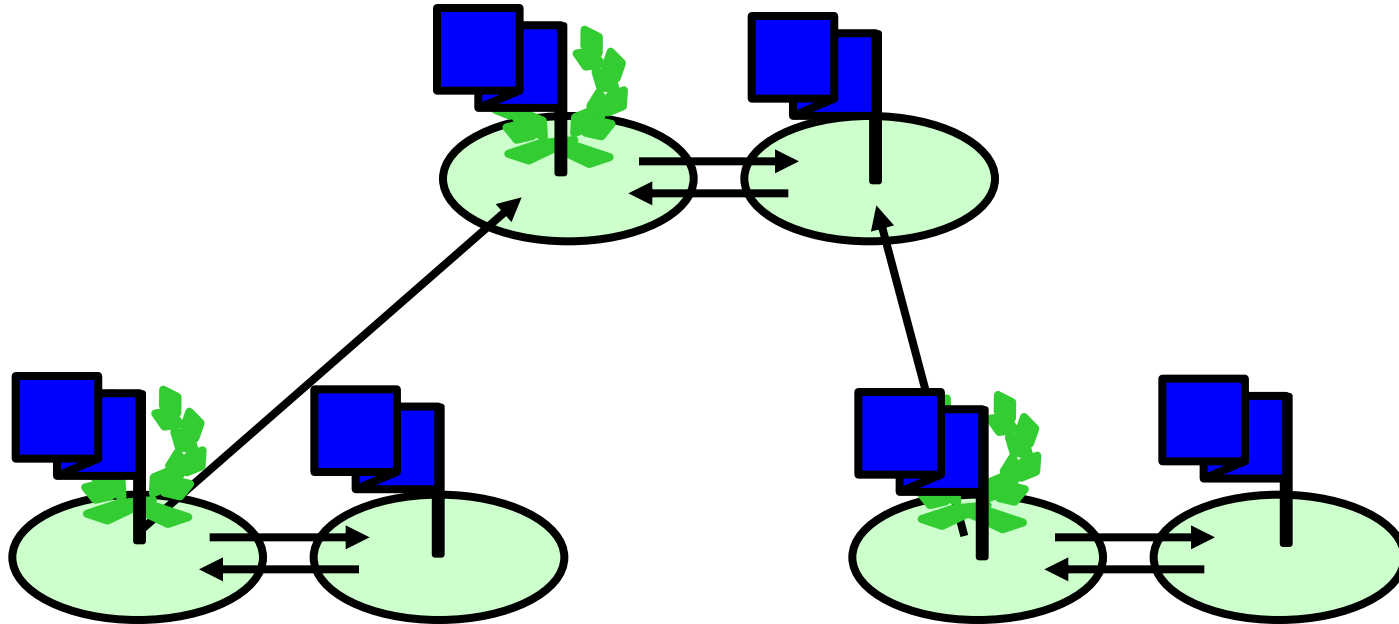
- If tree nodes have fan-in 2
 - Don't need to call `getAndDecrement()`
 - Winner chosen statically
- At level i
 - If i -th bit of `id` is 0, move up
 - Otherwise keep back



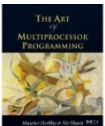
Tournament Tree Barriers



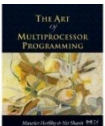
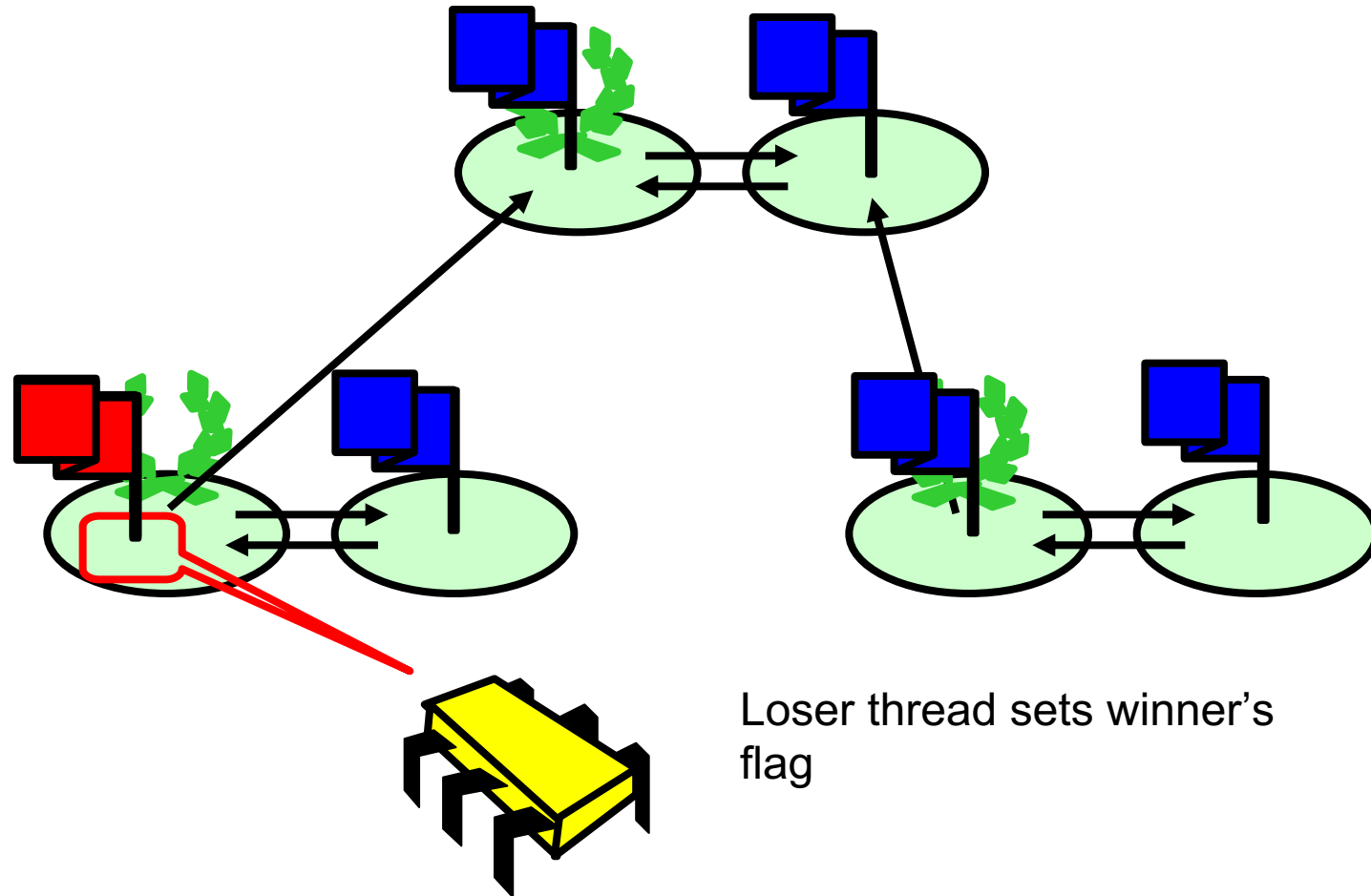
Tournament Tree Barriers



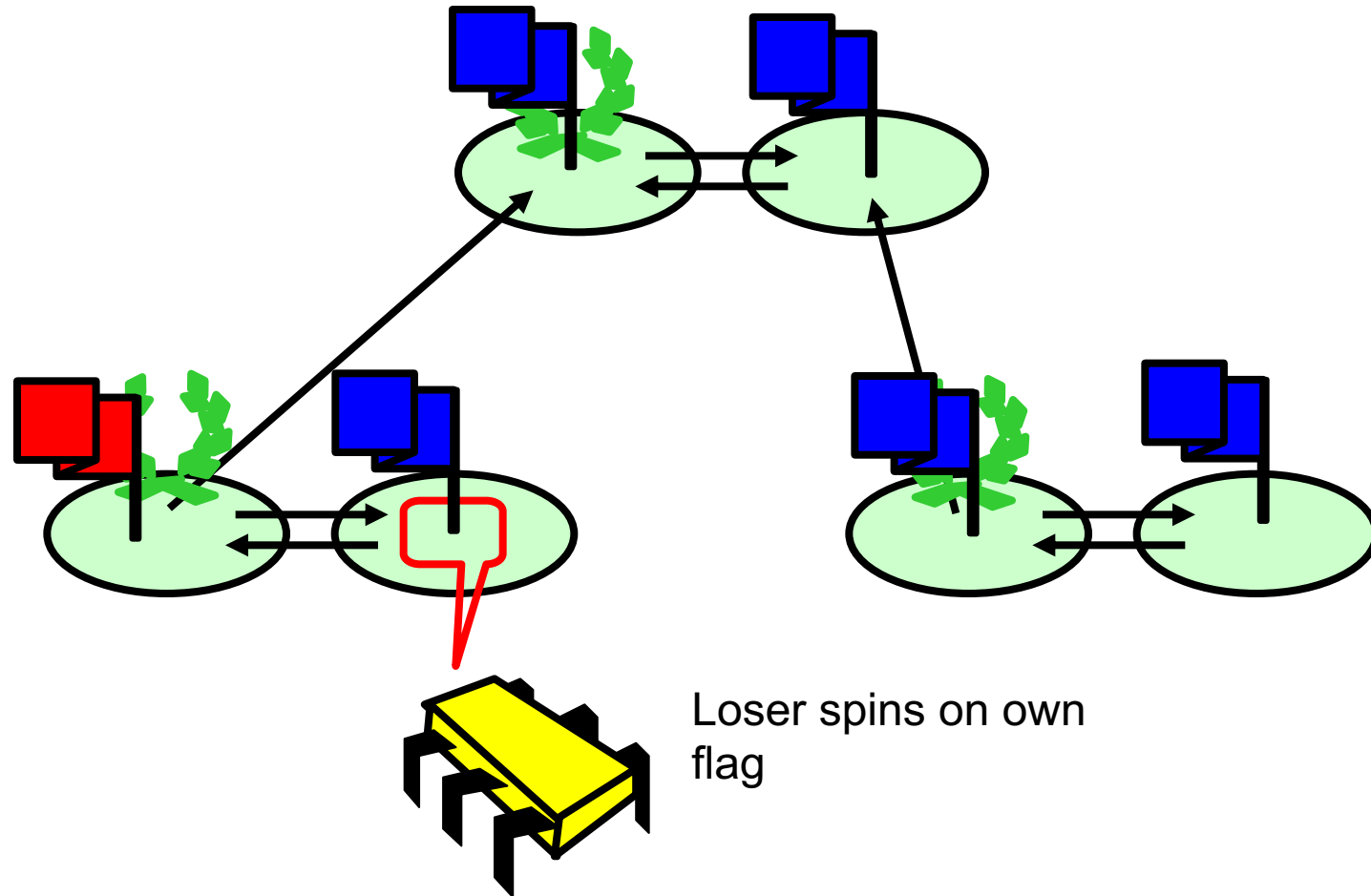
All flags blue



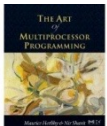
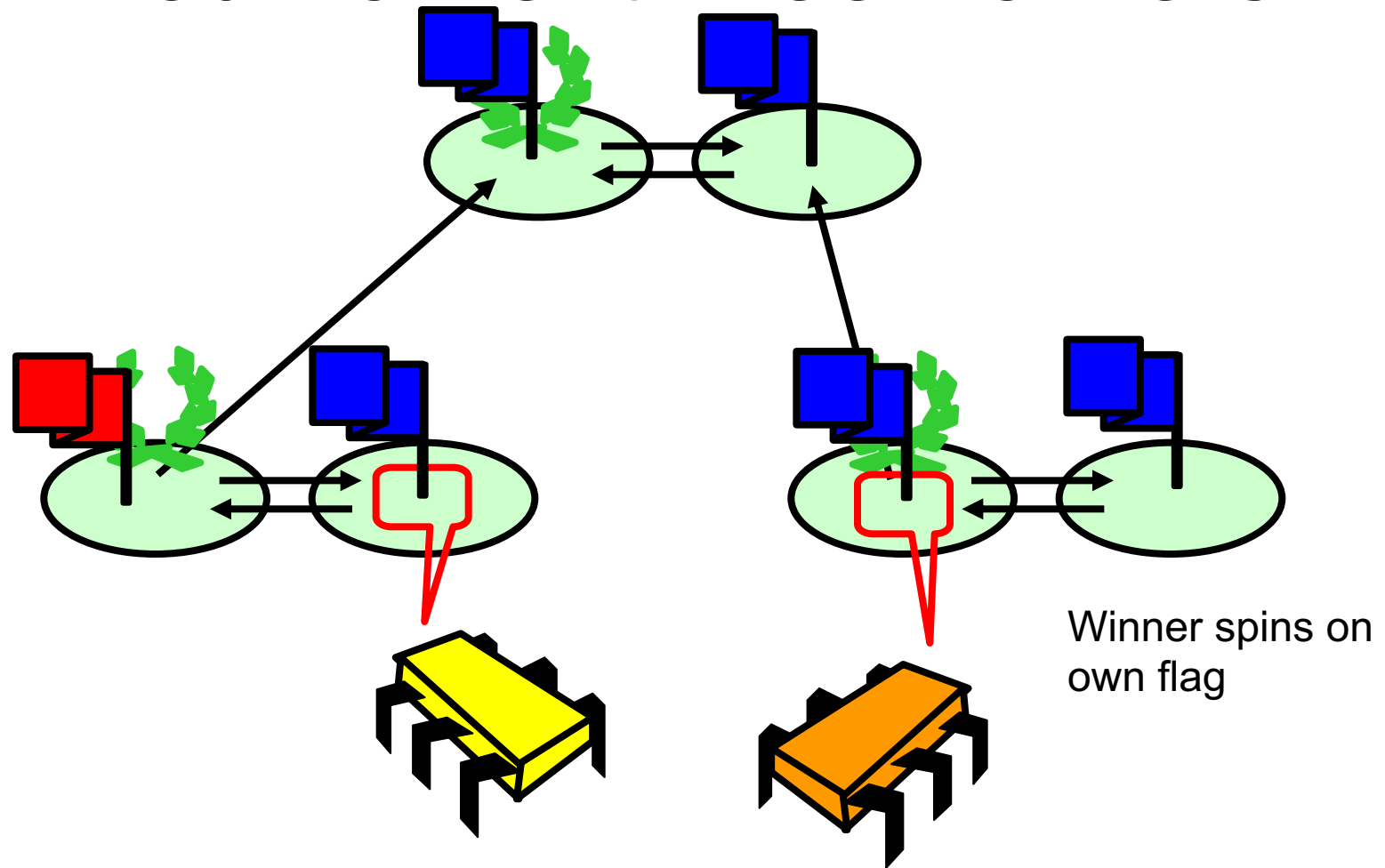
Tournament Tree Barriers



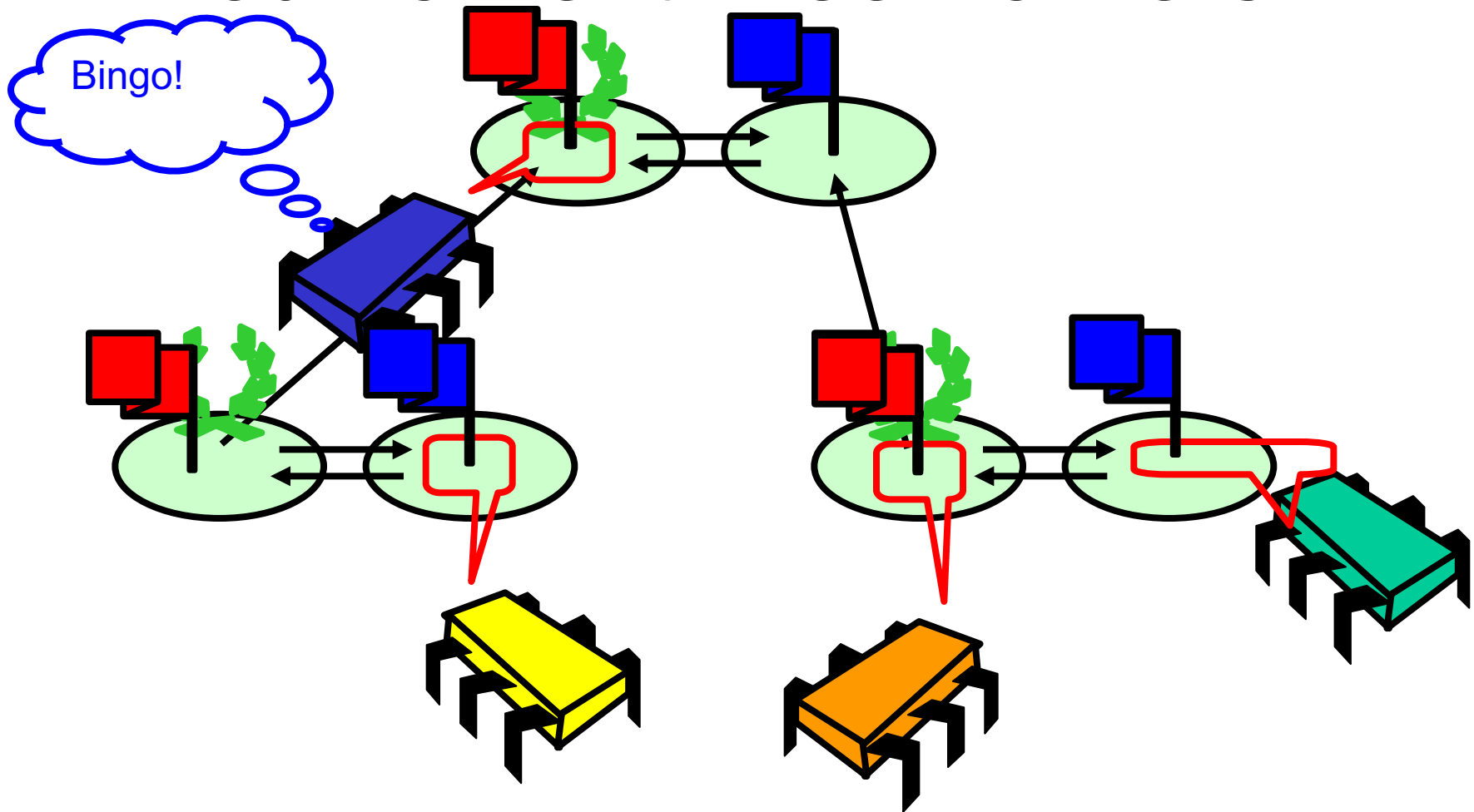
Tournament Tree Barriers



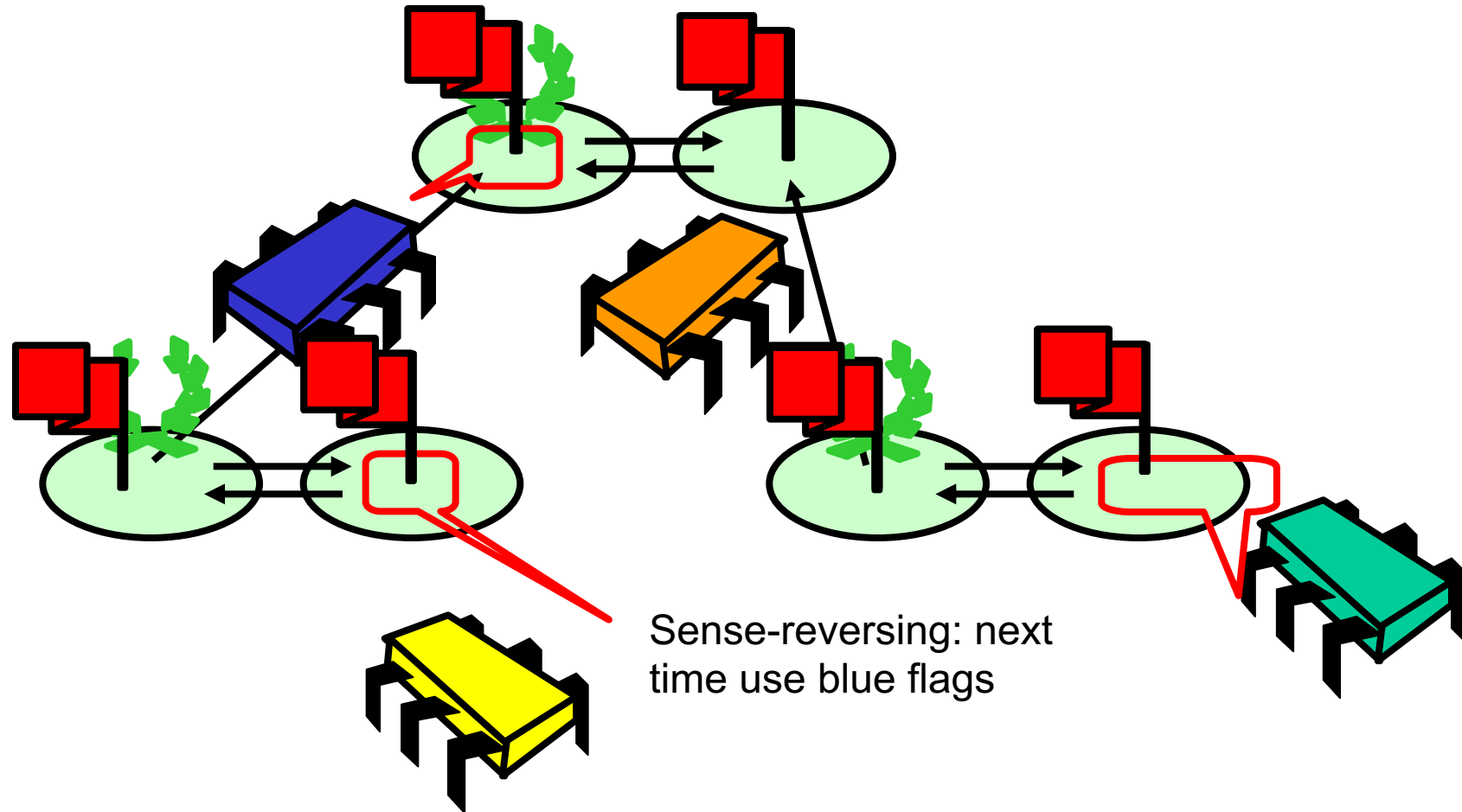
Tournament Tree Barriers



Tournament Tree Barriers

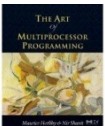


Tournament Tree Barriers



Tournament Barrier

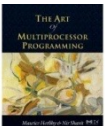
```
class TBarrier {  
    boolean flag;  
    TBarrier partner;  
    TBarrier parent;  
    boolean top;  
    ...  
}
```



Tournament Barrier

```
class TBarrier {  
    boolean flag;  
    TBarrier partner;  
    TBarrier parent;  
    boolean top;  
    ...  
}
```

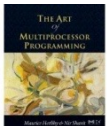
Notifications delivered here



Tournament Barrier

```
class TBarrier {  
    boolean flag;  
    TBarrier partner;  
    TBarrier parent;  
    boolean top;  
    ...  
}
```

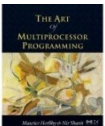
Other thread at same level



Tournament Barrier

```
class TBarrier {  
    boolean flag;  
    TBarrier partner;  
    TBarrier parent;  
    boolean top;  
    ...  
}
```

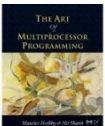
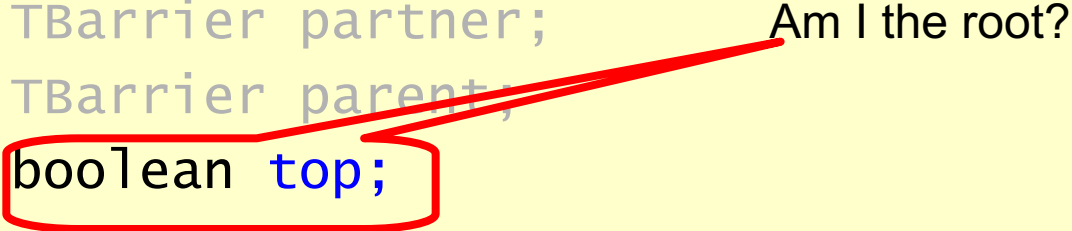
Parent (winner) or null
(loser)



Tournament Barrier

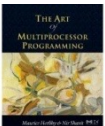
```
class TBarrier {  
    boolean flag;  
    TBarrier partner;  
    TBarrier parent;  
    boolean top;  
    ...  
}
```

Am I the root?



Tournament Barrier

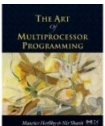
```
void await(boolean mySense) {  
    if (top) {  
        return;  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```



Tournament Barrier

Current sense

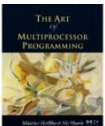
```
void await(boolean mySense) {  
    if (top) {  
        return; Le root, c'est moi  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```



Tournament Barrier

```
void await(boolean mySense) {  
    if (top) {  
        return;  
    }  
    else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```

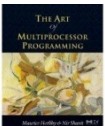
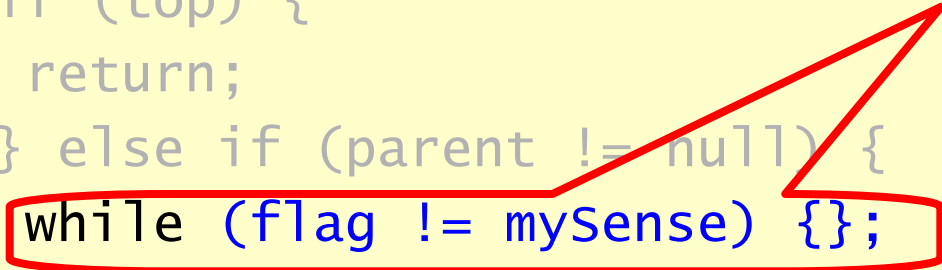
I am already a winner



Tournament Barrier

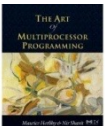
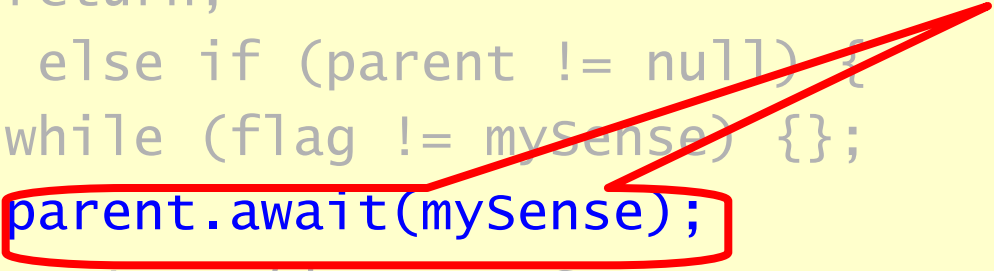
```
void await(boolean mySense) {  
    if (top) {  
        return;  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```

Wait for partner



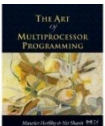
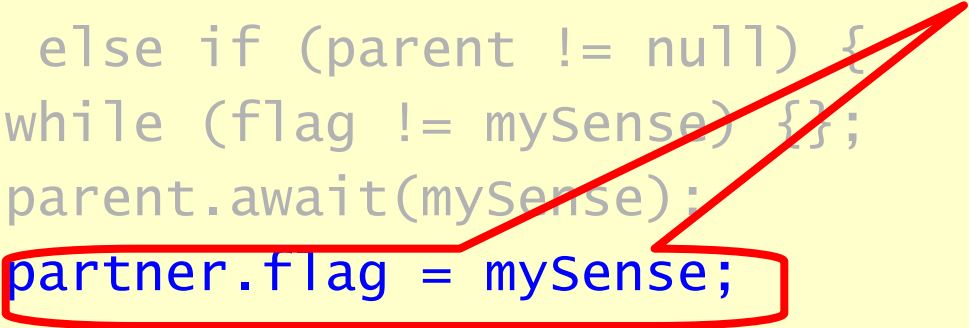
Tournament Barrier

```
void await(boolean mySense) {  
    if (top) {  
        return; Synchronize upstairs  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```



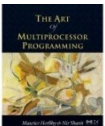
Tournament Barrier

```
void await(boolean mySense) {  
    if (top) {  
        return; Inform partner  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```



Tournament Barrier

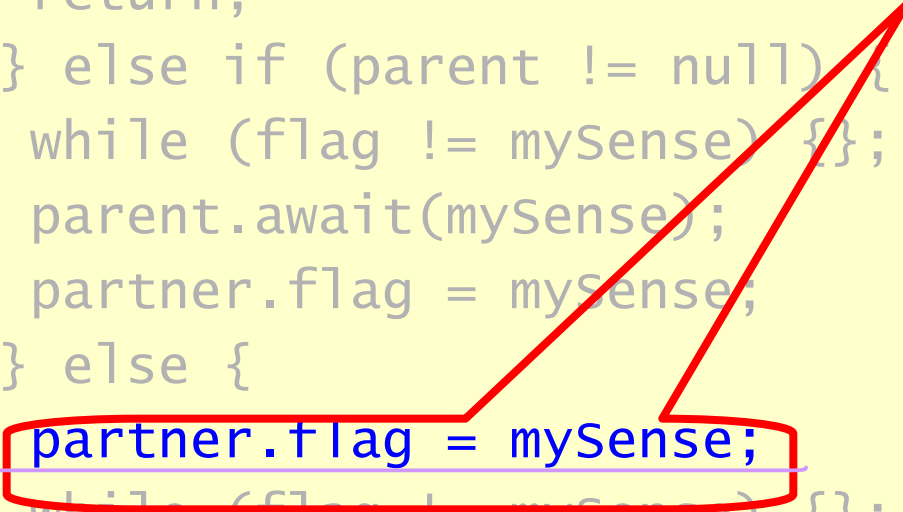
```
void await(boolean mySense) {  
    if (top) {  
        return; Natural-born loser  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```



Tournament Barrier

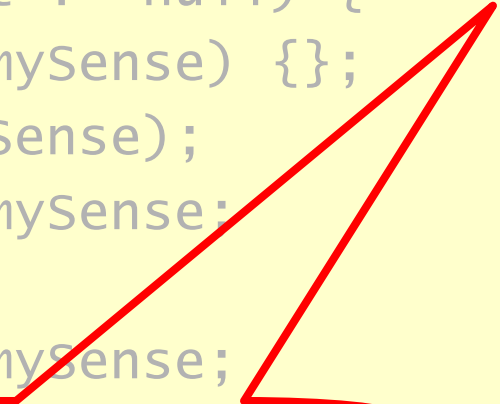
```
void await(boolean mySense) {  
    if (top) {  
        return;  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```

Tell partner I'm here



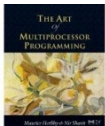
Tournament Barrier

```
void await(boolean mySense) {  
    if (top) {  
        return; Wait for notification from partner  
    } else if (parent != null) {  
        while (flag != mySense) {};  
        parent.await(mySense);  
        partner.flag = mySense;  
    } else {  
        partner.flag = mySense;  
        while (flag != mySense) {};  
    }  
}
```



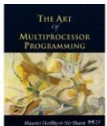
Remarks

- No need for read-modify-write calls
- Each thread spins on fixed location
 - Good for bus-based architectures
 - Good for NUMA architectures

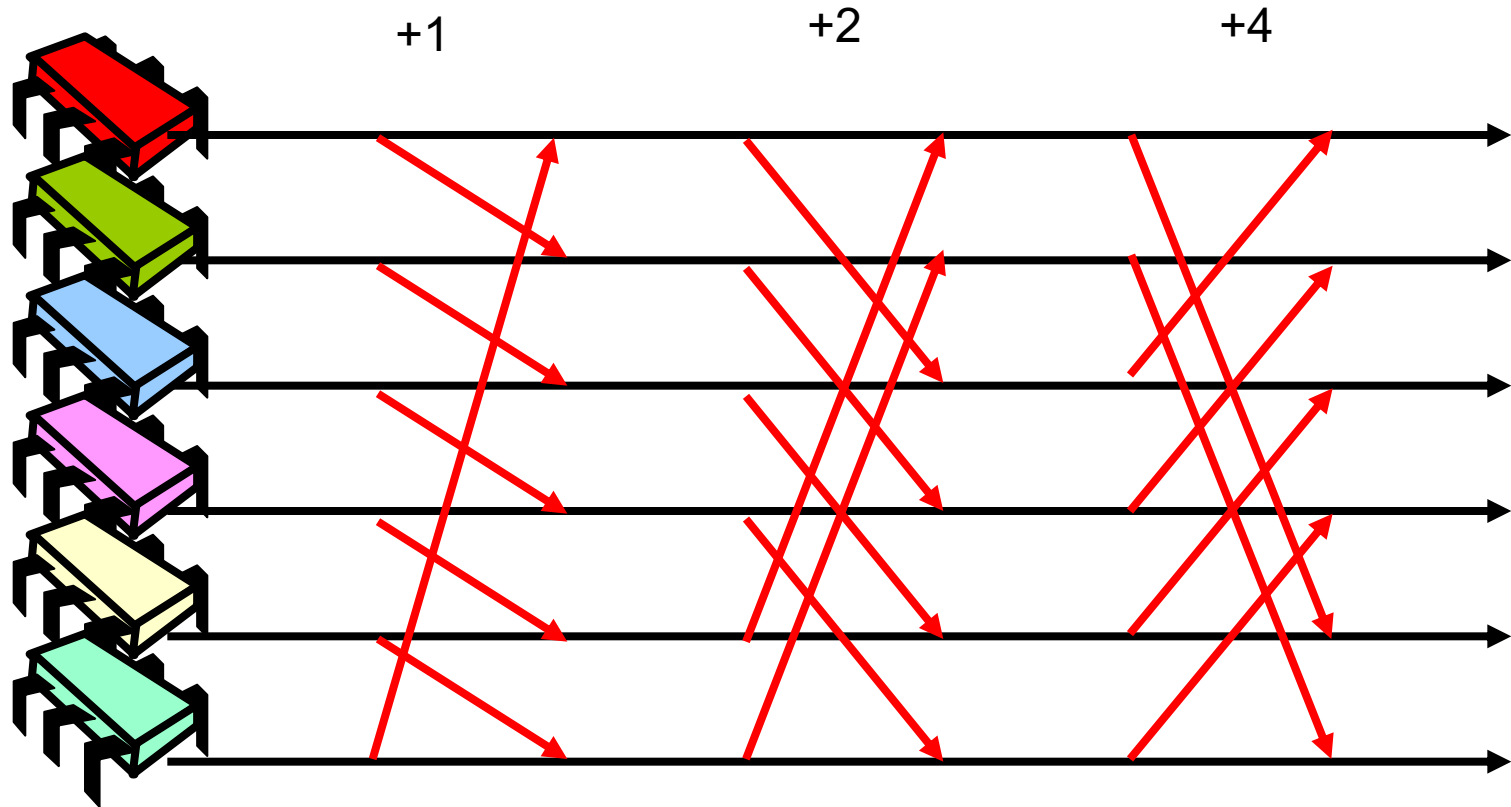


Dissemination Barrier

- At round i
 - Thread A notifies thread $A+2^i \pmod n$
- Requires $\log n$ rounds

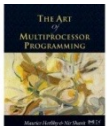


Dissemination Barrier



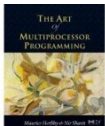
Remarks

- Elegant
- Good source of homework problems
- Not cache-friendly



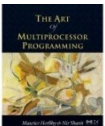
Ideas So Far

- Sense-reversing
 - Reuse without reinitializing
- Combining tree
 - Like counters, locks ...
- Tournament tree
 - Optimized combining tree
- Dissemination barrier
 - Intellectually Pleasing
(matter of taste)

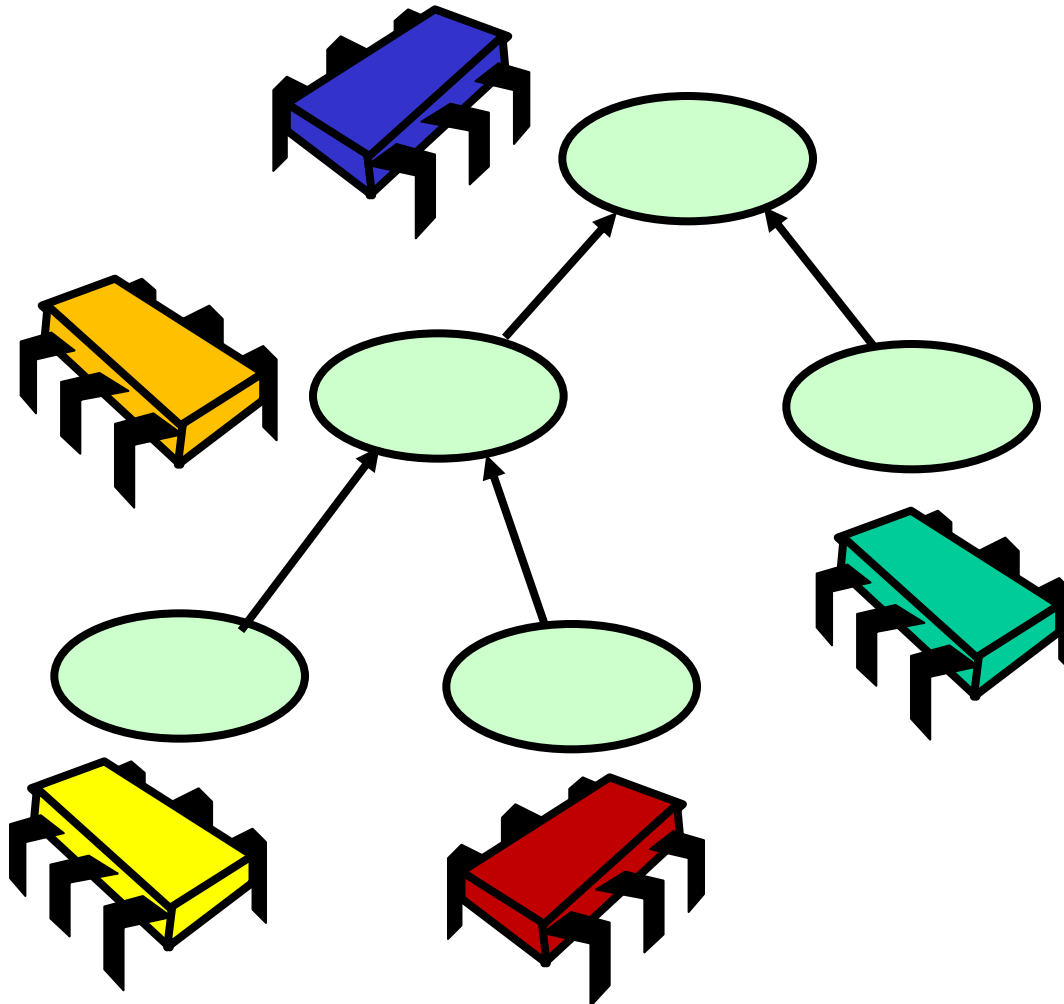


Which is best for Multicore?

- On a cache coherent multicore chip: perhaps none of the above...
- Here is another (arguably) better algorithm ...

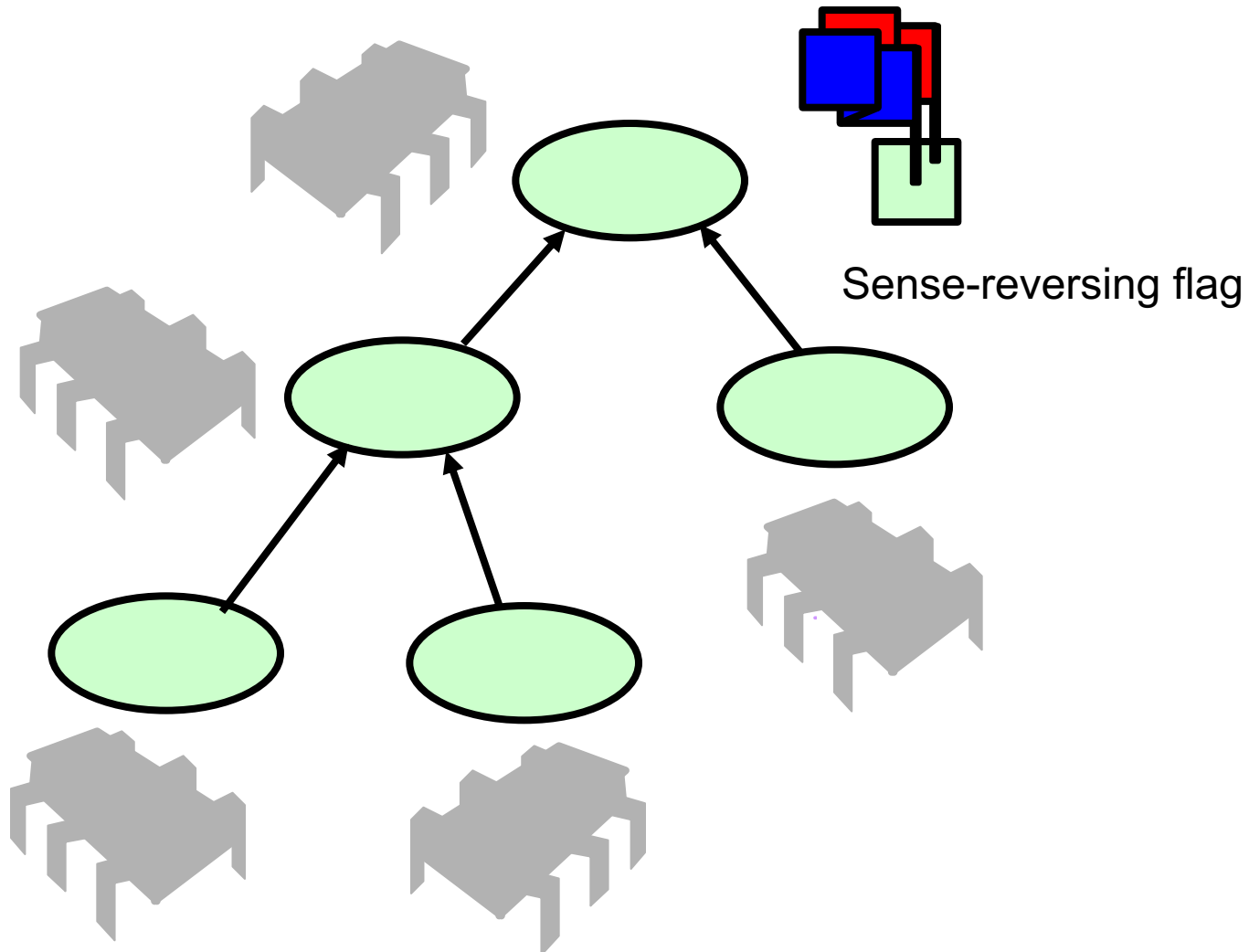


Static Tree Barrier

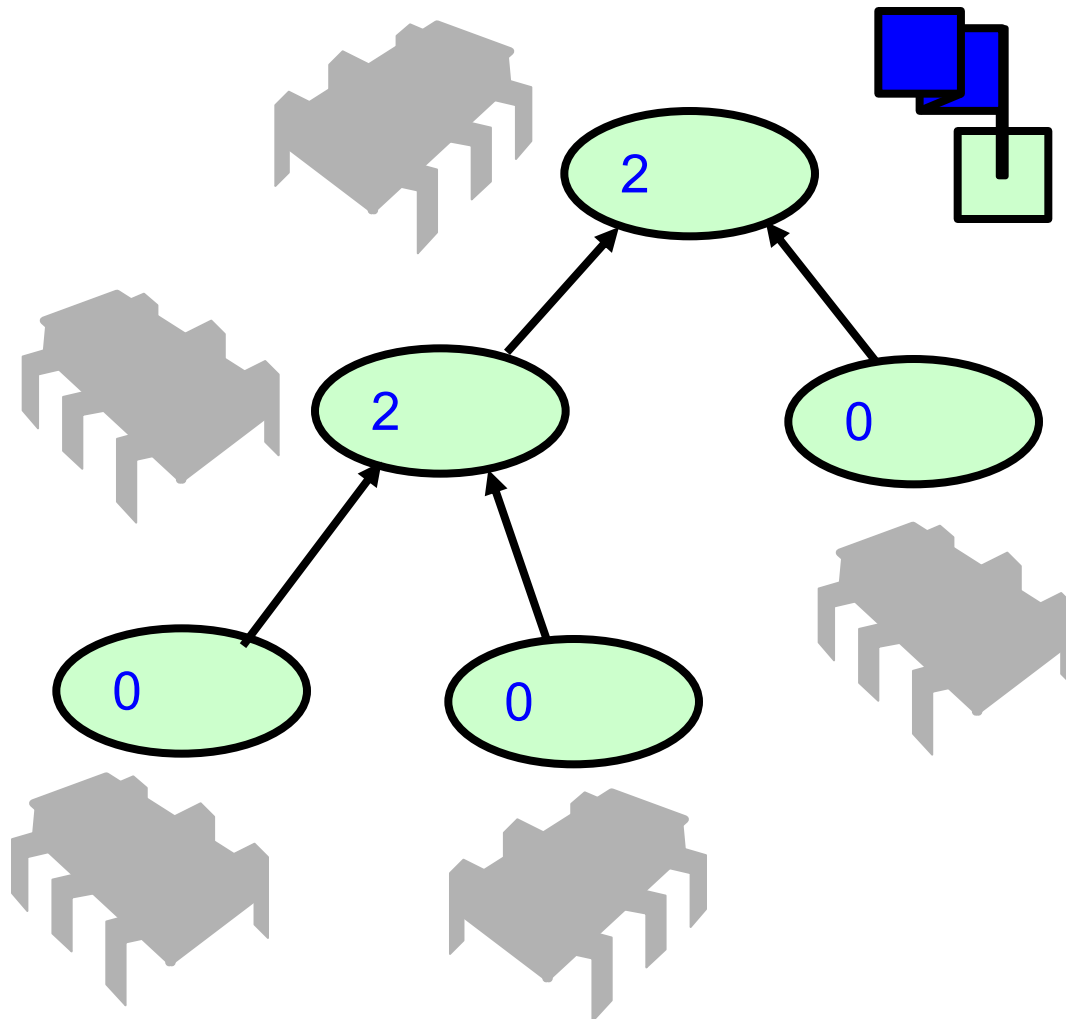


One node per thread,
statically assigned

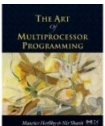
Static Tree Barrier



Static Tree Barrier

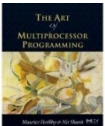
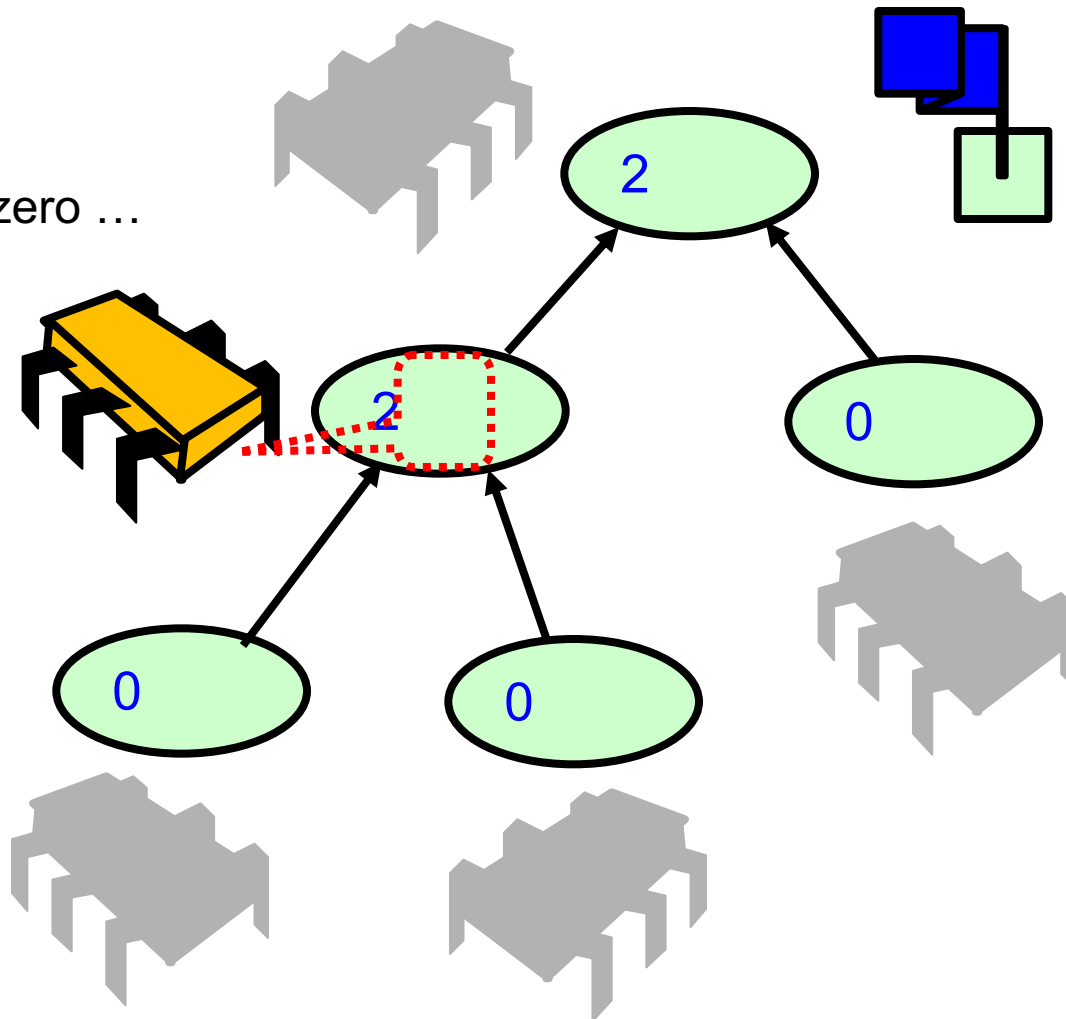


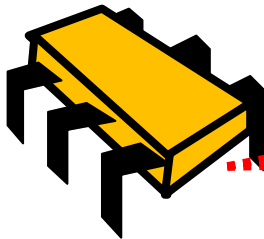
Node has count of missing children



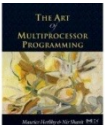
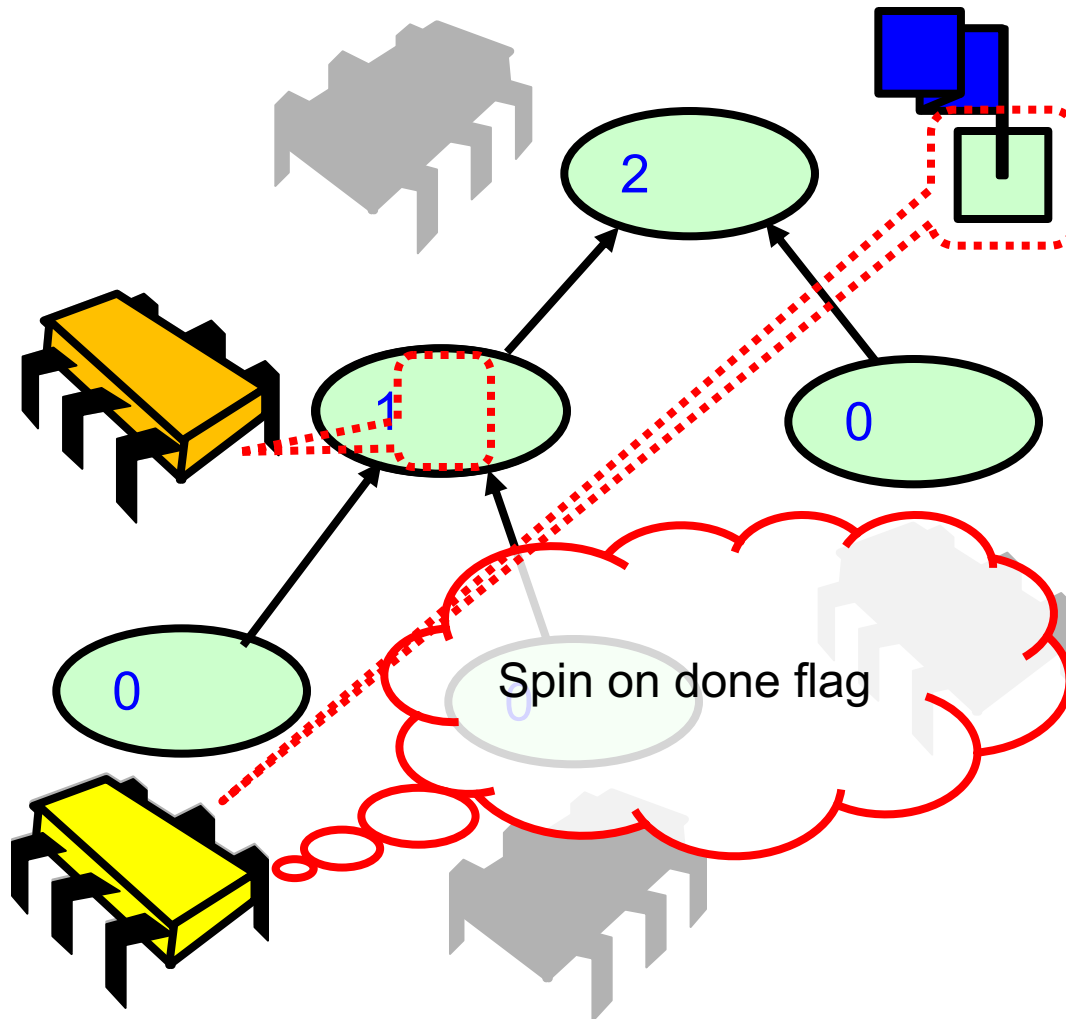
Static Tree Barrier

Spin until zero ...

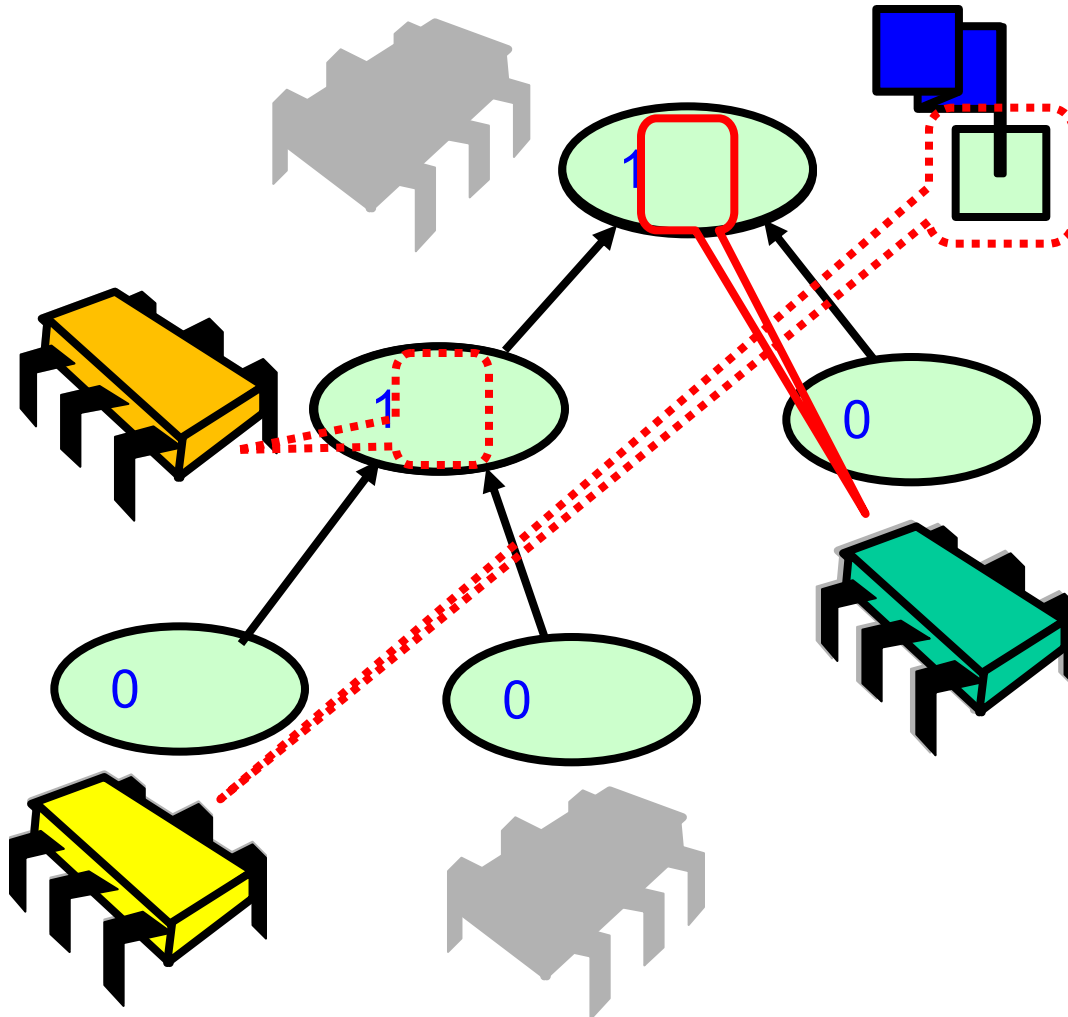




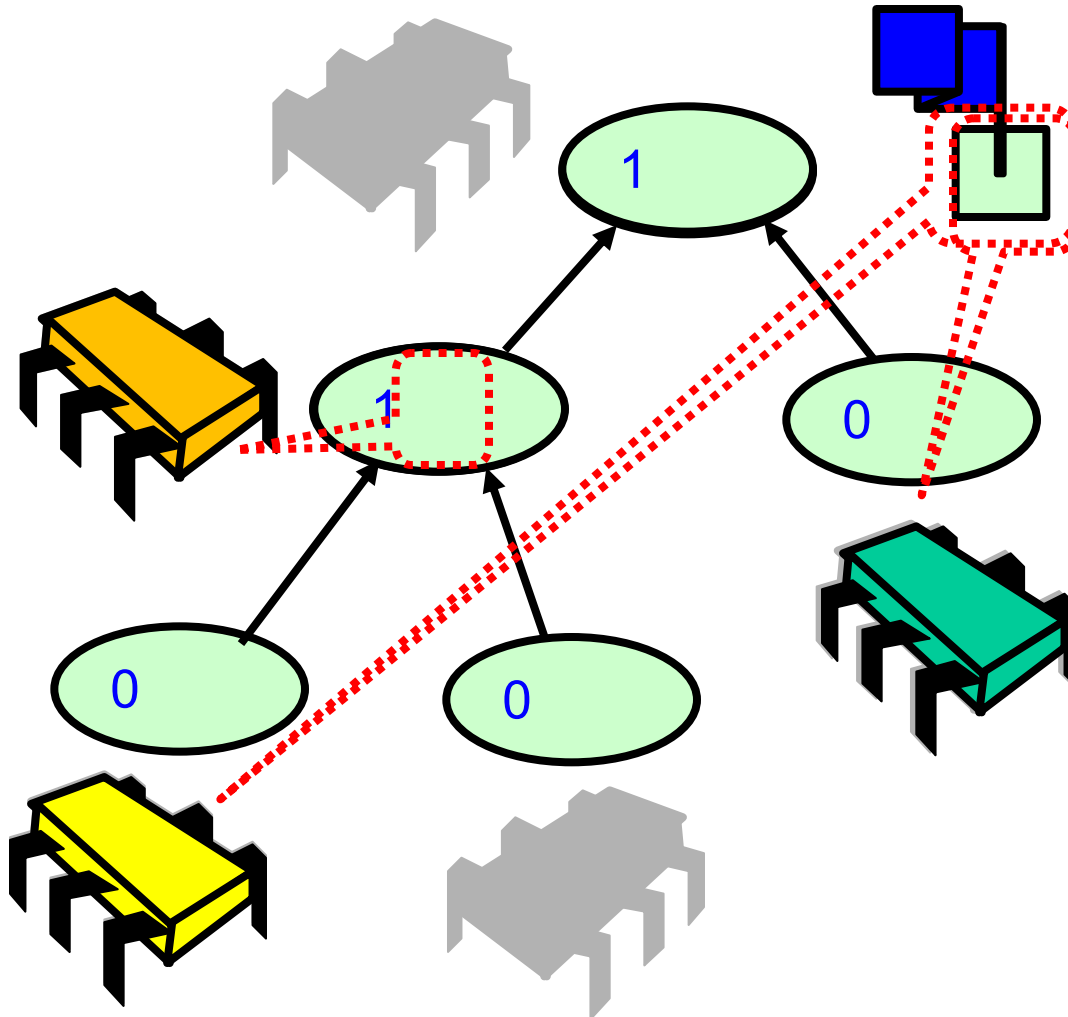
Static Tree Barrier



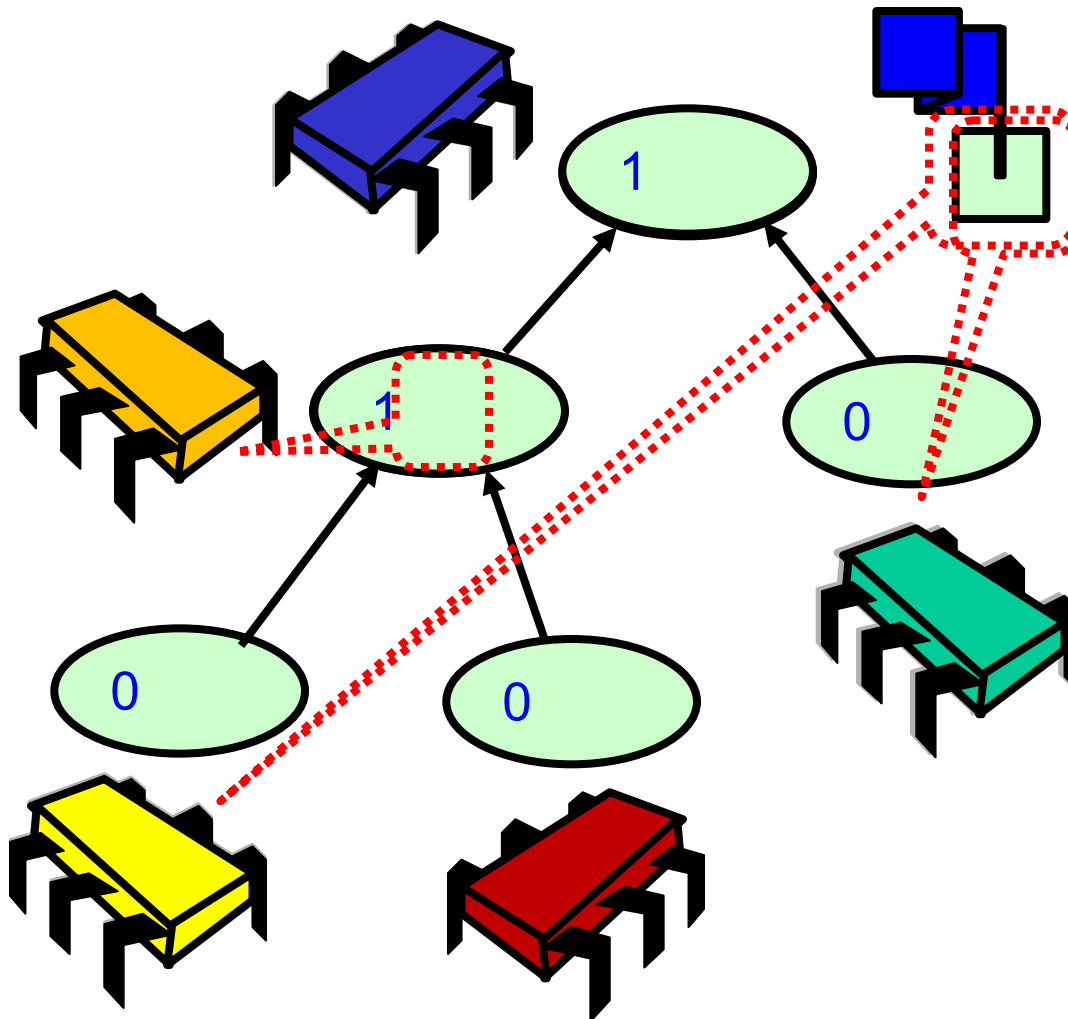
Static Tree Barrier



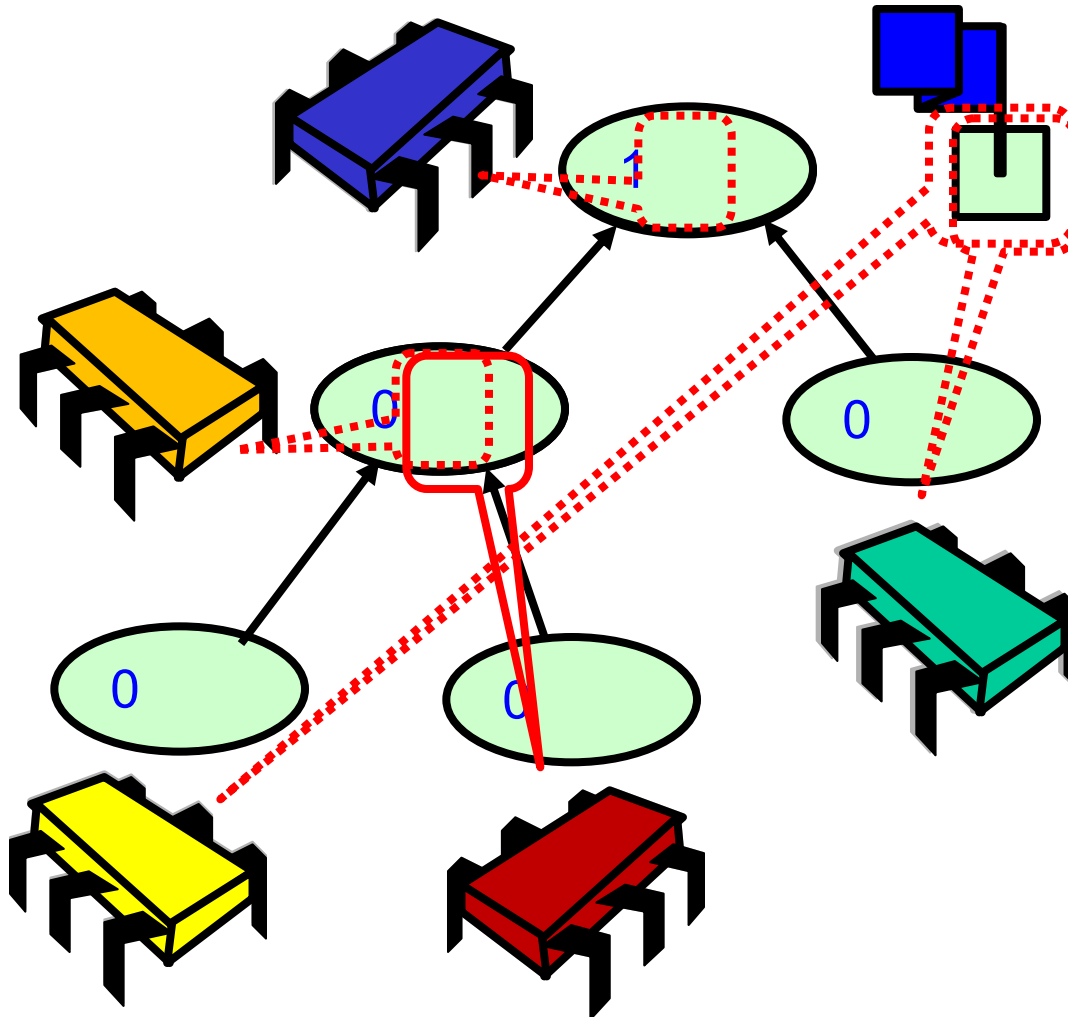
Static Tree Barrier



Static Tree Barrier

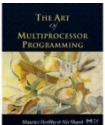
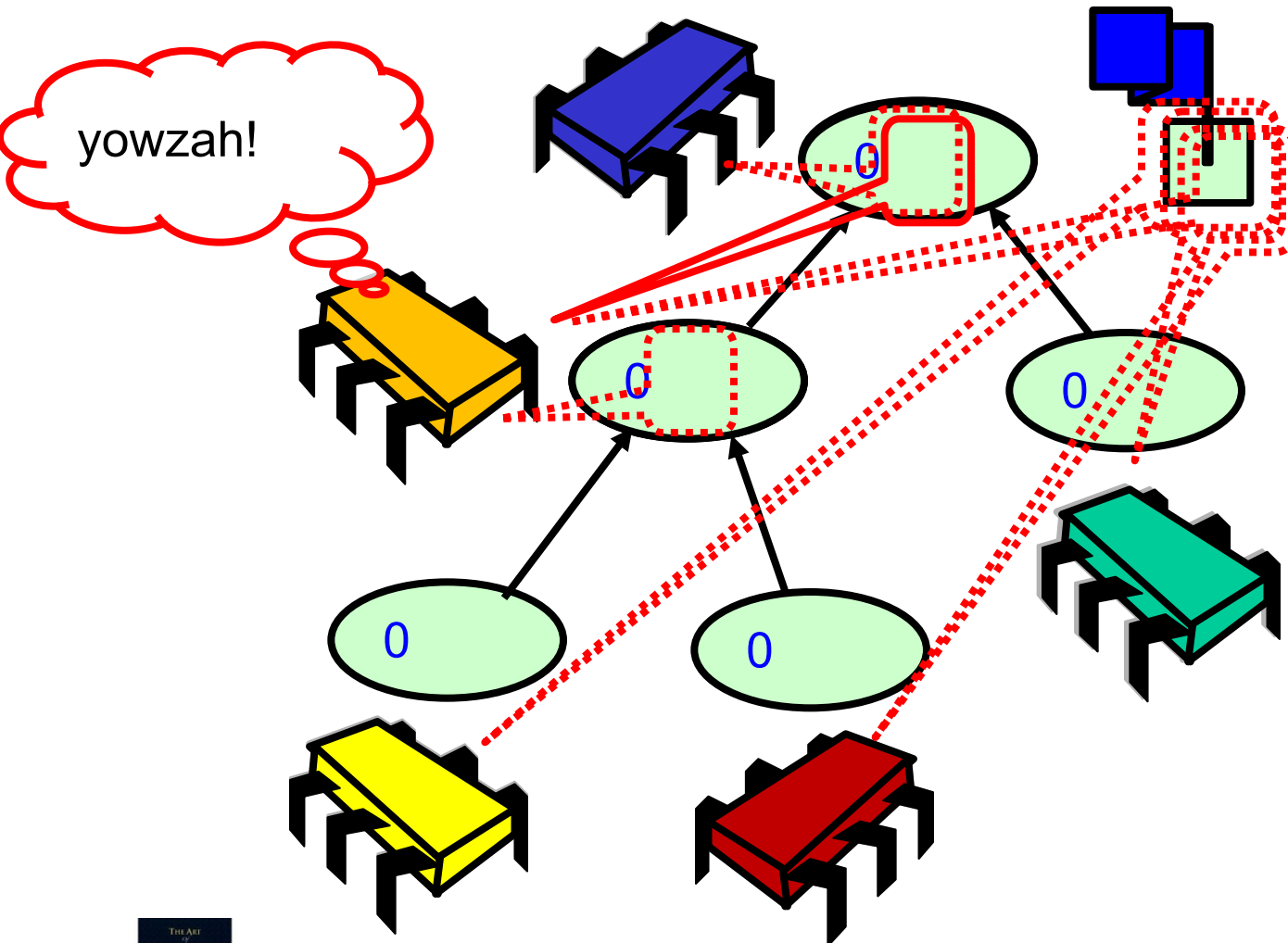


Static Tree Barrier



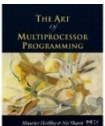
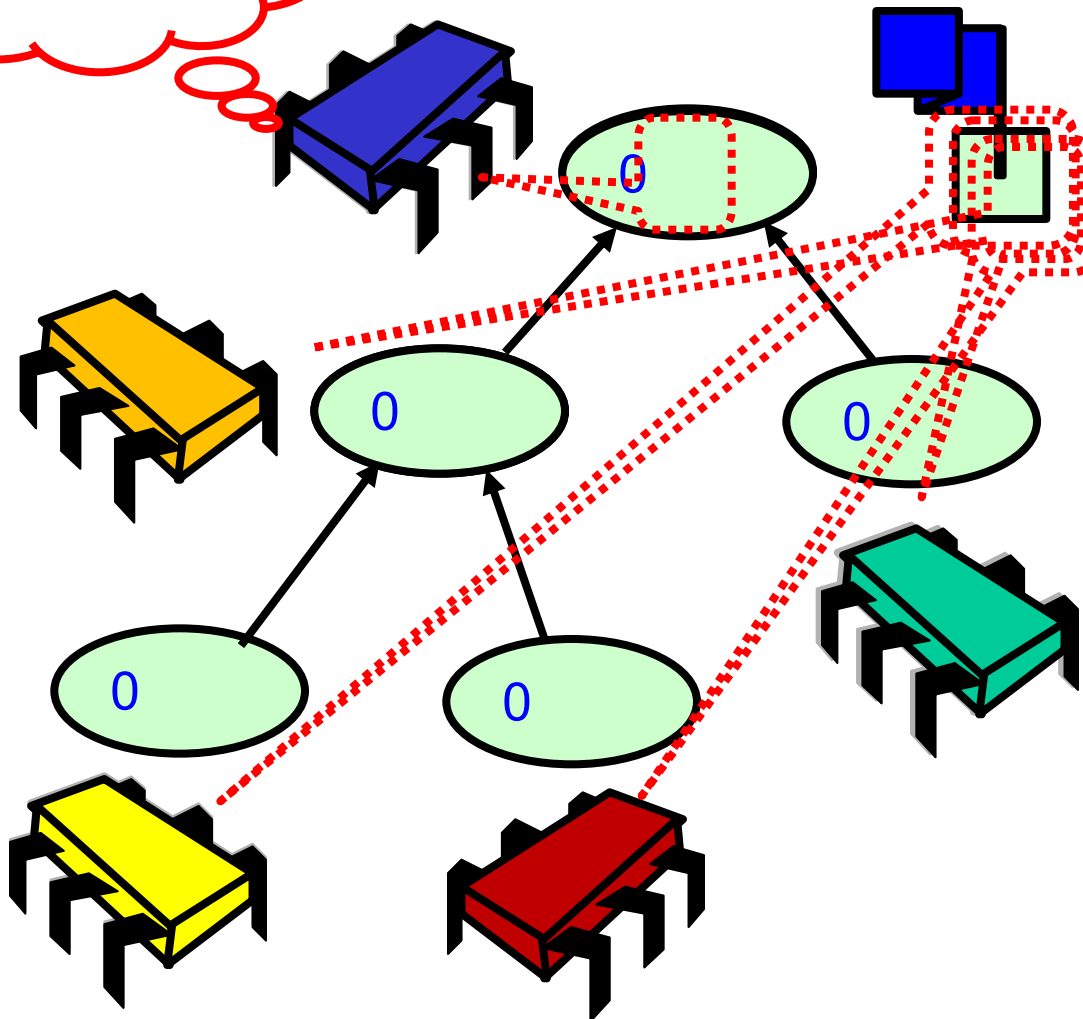


Static Tree Barrier



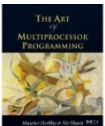
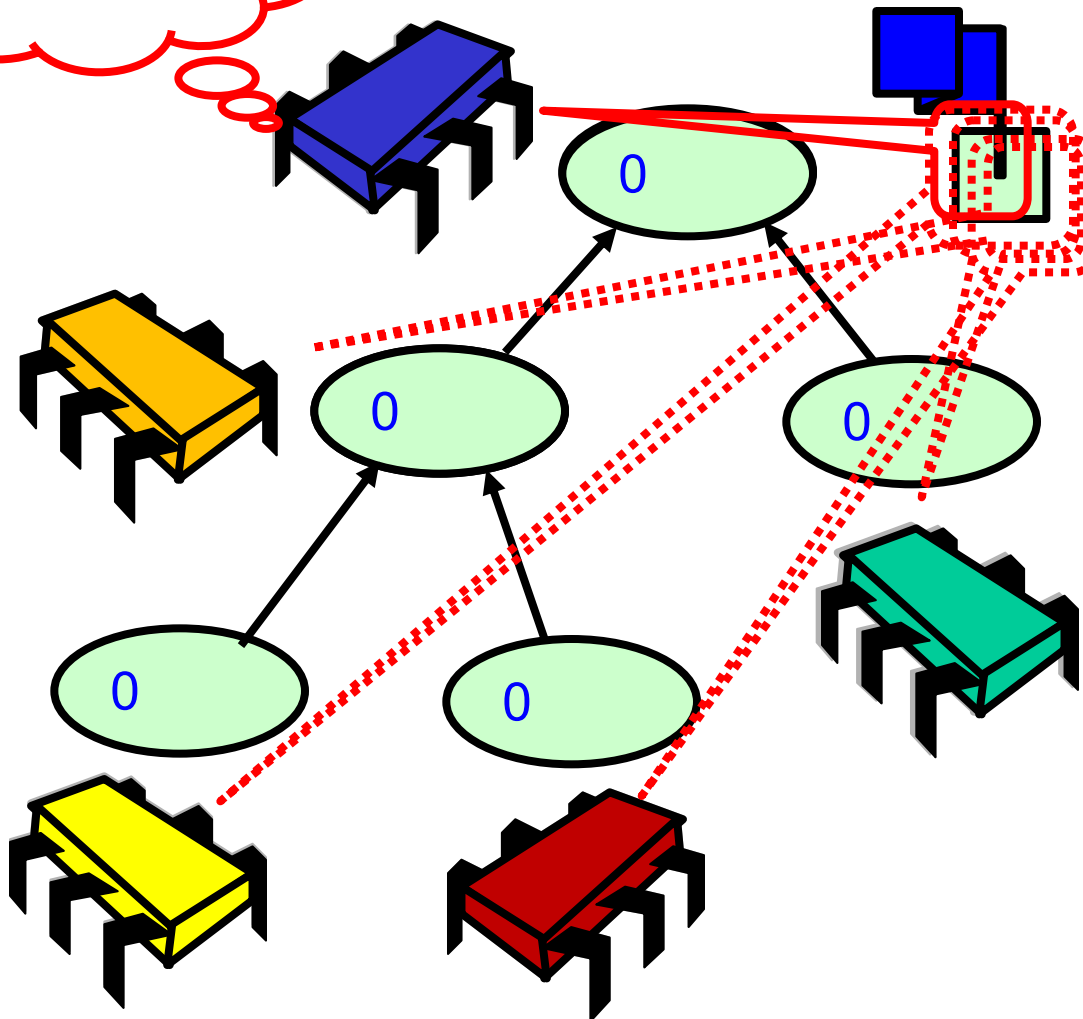
Static Tree Barrier

yowzah!

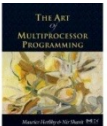
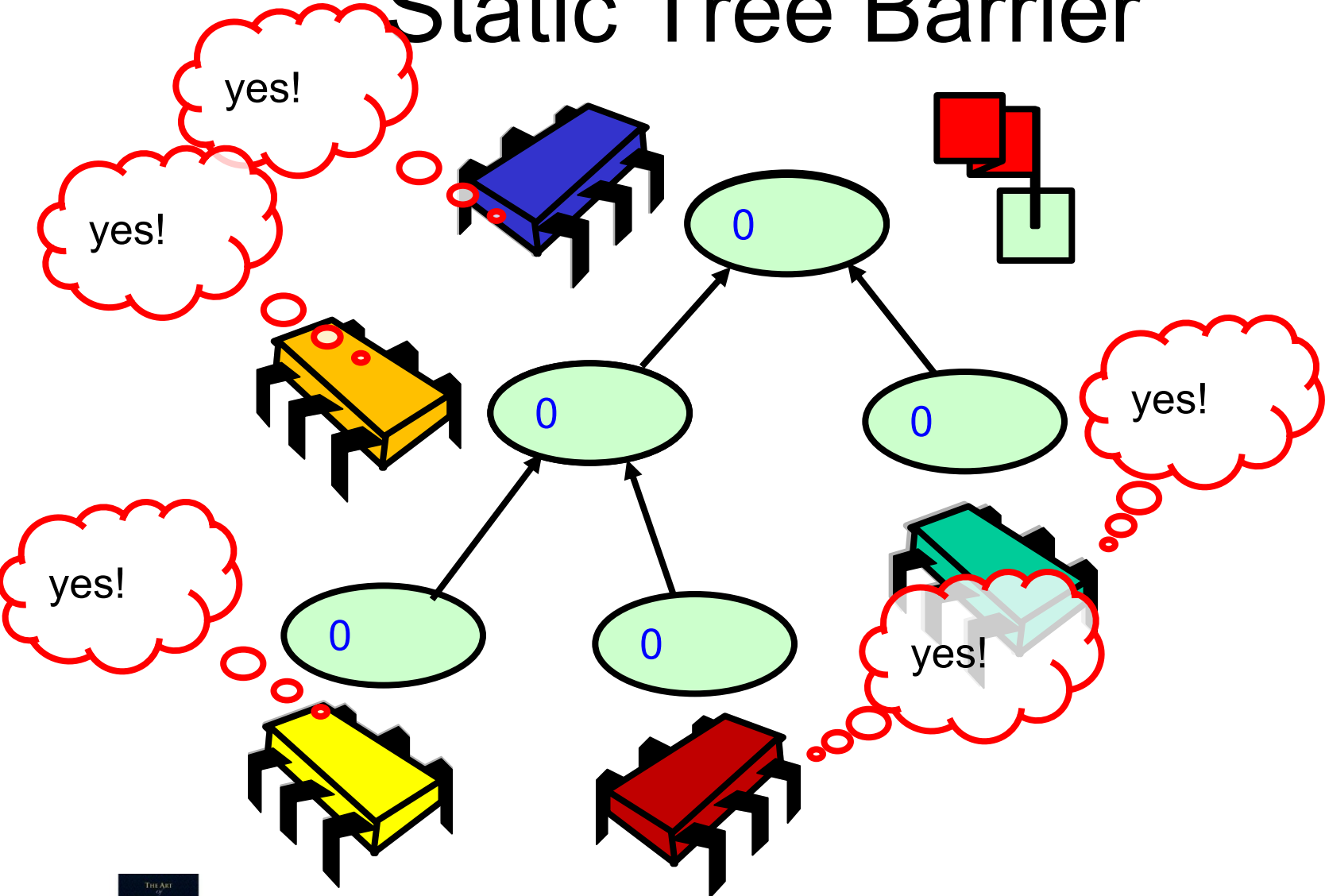


Static Tree Barrier

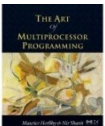
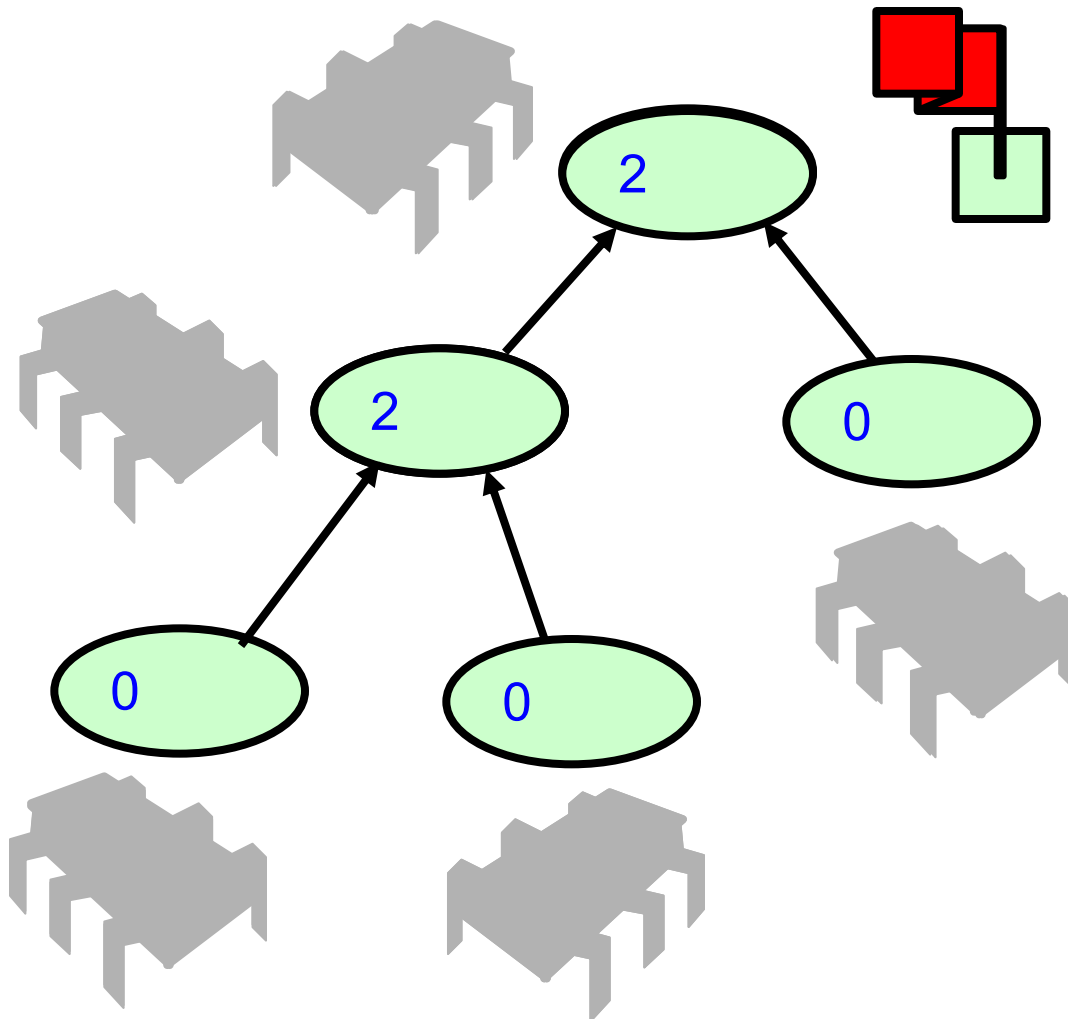
yowzah!



Static Tree Barrier

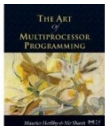


Static Tree Barrier



Remarks

- Very little cache traffic
- Minimal space overhead
- On message-passing architecture
 - Send notification & sense down tree



This work is licensed under a [Creative Commons Attribution-ShareAlike 2.5 License](https://creativecommons.org/licenses/by-sa/3.0/).

- You are free:
 - to Share — to copy, distribute and transmit the work
 - to Remix — to adapt the work
 - Under the following conditions:
 - Attribution. You must attribute the work to “The Art of Multiprocessor Programming” (but not in any way that suggests that the authors endorse you or your use of the work).
 - Share Alike. If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.
- For any reuse or distribution, you must make clear to others the license terms of this work. The best way to do this is with a link to
 - <http://creativecommons.org/licenses/by-sa/3.0/>.
- Any of the above conditions can be waived if you get permission from the copyright holder.
 - Nothing in this license impairs or restricts the author's moral rights.